CAN FAILED TRAJECTORIES REVEAL WHERE AGENTS GO WRONG? AGENTIC FAILURE ATTRIBUTION WITHOUT STEP-LEVEL LABELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Failure attribution for LLM agents aims to identify the decisive step responsible for task failure, which is critical for debugging and improving agentic systems. Existing methods rely on costly LLM prompting, step-level error annotations, or verified successful trajectories that lack failure signals for assistance. In this paper, we study failure attribution from *step-unlabeled failure trajectories*, where each trajectory is known to have failed but its decisive step is unknown. We propose SOAP (**S**pectral sc**O**ring with **A**ttention-guided **P**ropagation), a novel framework requiring neither step-level training labels nor a separate success-only reference set. SOAP first constructs a spectral reference space from pooled step representations and assigns each step a base error score according to its projection behavior. Importantly, later steps may exhibit stronger failure signals because they inherit an earlier mistake, causing independent scorers to favor downstream consequences over the original error. To address this *downstream contamination*, SOAP derives soft step dependencies from attention patterns and propagates downstream error evidence toward its likely upstream sources. Extensive experiments show that SOAP consistently improves attribution performance over competitive baselines.

1 INTRODUCTION

When an LLM-based agent fails on a long-horizon task, identifying *where the failure began* is essential for debugging, auditing, and improving the agent. An agent trajectory may contain dozens or even hundreds of reasoning, communication, and tool-use steps, while only one early decision introduces the error that ultimately causes task failure. Manually tracing such trajectories is prohibitively expensive and prone to inconsistency, especially as agents are increasingly deployed for software engineering, scientific discovery, and other complex tasks (Yao et al., 2023; Wang et al., 2024). This motivates the problem of *failure attribution*: given a failed trajectory, identify the decisive step that introduced the failure (Zhang et al., 2025c).

Failure attribution is fundamentally challenging. Existing methods typically rely either on prompting powerful LLM judges, which incurs substantial inference cost and can be unreliable over long trajectories (Zhang et al., 2025c; Banerjee et al., 2025; Wang et al., 2026; Rafi et al., 2026; Yu et al., 2025), or on post-training attribution models using failure trajectories with step-level annotations (Zhang et al., 2025a;b). Such annotations are costly and often ambiguous, since identifying the decisive step may require substantial domain expertise. OAT (Yeh et al., 2026) alleviates this burden by learning the latent dynamics of successful trajectories and detecting deviations from the learned flow. However, it has no access to failure trajectories during training, and therefore *models only what successful behavior looks like*. Moreover, its success-only reference set must be verified and may become stale as agents and task distributions evolve. This motivates a complementary question:

Can failed trajectories themselves enable failure attribution without step-level annotations?

Failed trajectories provide a complementary source of supervision. In many agentic systems, whether a task has failed can be determined from execution outcomes or task evaluators, while identifying the step responsible for that failure remains costly. We therefore consider a collection of *step-unlabeled failure trajectories*, i.e., every trajectory is known to have failed, but no step is annotated as decisive or non-decisive. Each trajectory consequently contains an unlabeled mixture

of ordinary steps and the decisive error. Unlike OAT (Yeh et al., 2026), such a failure-conditioned dataset directly exposes the behaviors that arise when agents fail. Yet harnessing this information is non-trivial: without step-level membership labels, the learner must extract a meaningful failure signal without knowing which step introduced the failure. The first challenge is therefore to *identify decisive-error evidence from a step-level unlabeled mixture within failed trajectories*.

A second challenge is that the step exhibiting the strongest failure signal need not be the step that introduced the failure. Agent trajectories are autoregressive because each step is generated from preceding observations and actions. Once an error occurs, later steps operate on a corrupted context and may produce failed tool calls, contradictory reasoning, or repeated recovery attempts. These downstream consequences can be more conspicuous than the original error. For example, a plausible but incorrect tool argument may trigger a sequence of visible error messages and unsuccessful retries, causing a conventional per-step scorer to select the final retry rather than the initial incorrect action. We refer to this phenomenon as *downstream contamination* (Figure 1). Recent methods reconstruct dependencies through LLM prompting or specialized training, then perform graph search or backtracking (Rafi et al., 2026; Wang et al., 2026; Zhang et al., 2025b), which introduce substantial inference or training overhead. This motivates a lightweight alternative that extracts soft dependencies from the model to separate root causes from downstream effects.

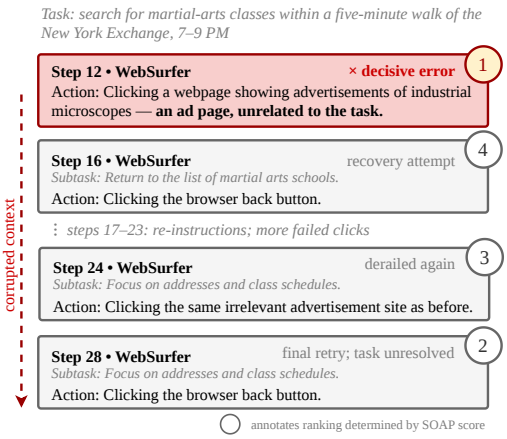


Figure 1: *Downstream contamination* in a failed Who&When trajectory. An early wrong click (step 12) lands the browser on an irrelevant page; later steps are spent on recovery attempts that derail again.

In this paper, we propose SOAP (**S**pectral sc**O**ring with **A**ttention-guided **P**ropagation), a framework for decisive-error localization from step-unlabeled failure trajectories. At a high level, SOAP consists of two components. **First**, it constructs a *spectral reference space* from step representations extracted from the failure trajectories. Each step receives a base error score according to its projection onto a selected band of singular directions. This produces a step-level signal without requiring decisive-error annotations or a separate collection of verified successful trajectories. **Second**, SOAP addresses downstream contamination through *dependency-aware rescoring*. We derive a soft dependency graph from models' attention patterns, where each edge approximates how strongly a later step draws on an earlier one. Rather than treating downstream failure signals as independent and competing evidence, SOAP propagates them backward toward the preceding steps on which they depend. An earlier step is therefore promoted when anomalous downstream behavior is systematically built upon it.

We evaluate SOAP on [TODO: benchmarks] using [TODO: agent systems and proxy models]. Across [TODO: number] evaluation settings, SOAP consistently improves decisive-step localization over different competitive baselines. In particular, SOAP improves [TODO: primary metric] by [TODO: result] over the strongest baseline on [TODO: benchmark]. Ablation studies further show that both spectral step scoring and attention-guided rescoring contribute to the improvements, which cannot be explained by simply favoring earlier steps or uniformly aggregating downstream scores. [TODO: see my comments in the exp section.] Our main contributions are summarized as follows:

- We formulate decisive-error localization from *step-unlabeled trajectories* within failed trajectories. This setting exploits trajectory-level failure signals while requiring neither decisive-error annotations nor a carefully verified successful trajectories.
- We identify *downstream contamination* as a central obstacle to decisive-error localization, i.e., later steps may exhibit stronger failure signals because they inherit an earlier mistake, causing independent per-step scorers to systematically localize failures too late.
- We propose SOAP, a novel framework performing spectral step scoring with attention-derived dependency propagation. Extensive experiments and ablations on [TODO: benchmarks] demonstrate when and why SOAP improves attribution. The results provide insights into leveraging step-unlabeled failure trajectories for agentic failure attribution.

2 PROBLEM SETUP

We formalize the task of localizing the decisive error in a failed agent trajectory.

Agent trajectory. Let x denote a task description and let $\tau = (s_1, \ldots, s_T)$ be the resulting trajectory of a multi-agent system. Each step s_t contains the action or message produced at turn t and is associated with an agent $a_t \in \{1, \ldots, K\}$. For evaluation, a failed trajectory is paired with a gold decisive-error index $t^\star \in \{1, \ldots, T\}$, corresponding to the earliest step at which the error responsible for the eventual task failure is introduced. The responsible agent is therefore $a_{t^\star}$.

Our framework extracts step representations and attention patterns using a frozen language model $\mathcal{M}$. When the model that generates the trajectory exposes its internal states, $\mathcal{M}$ can be the generating model itself. Otherwise, $\mathcal{M}$ can be a separate open-weight proxy model, allowing the framework to analyze trajectories produced by proprietary agents whose internals are inaccessible. In either case, $\mathcal{M}$ is used without fine-tuning on failure-attribution data.

Failure attribution. Given the task description x and a failed trajectory τ, the goal of failure attribution is to estimate the decisive-error step (Zhang et al., 2025c). The learner predicts an index $\hat{t}$ as an estimator of $t^\star$, with the corresponding responsible-agent prediction $\hat{a} = a_{\hat{t}}$.

Rather than predicting the decisive-error index directly, we assign each step a scalar attribution score $\widetilde{S}(s_t) \in \mathbb{R}$, where a larger value indicates that the step is more likely to have introduced the failure. The top-1 prediction is $\hat{t} = \arg\max_{t \in \{1,\ldots,T\}} \widetilde{S}(s_t)$. Sorting the scores further produces a ranked list of candidate steps for failure triage. We evaluate both top-1 localization and ranked-set performance in §4.

Step-unlabeled failure trajectories. In addition to the test trajectory, we assume access to a reference collection of failed trajectories $\mathcal{D}_{\text{fail}} = \{\tau^{(n)}\}_{n=1}^{N}, \tau^{(n)} = (s_1^{(n)}, \ldots, s_{T_n}^{(n)})$. The task-level outcome of each trajectory is known: every trajectory in $\mathcal{D}_{\text{fail}}$ ends in failure. However, no step-level annotation is available to identify which step introduced the failure or which agent was responsible. We therefore model the pooled steps in $\mathcal{D}_{\text{fail}}$ as an unlabeled contamination mixture (Huber, 1964)

$$P_{\text{fail}} = (1 - \pi) P_{\text{ord}} + \pi P_{\text{err}}, \tag{1}$$

where P_{ord} denotes the marginal distribution of ordinary steps, P_{err} denotes that of decisive-error steps, and $\pi \in (0, 1)$ is an unknown contamination proportion. Equivalently, each trajectory has an unobserved error set $\mathcal{E}^{(n)} \subseteq \{1, \ldots, T_n\}$, typically containing a single decisive step, such that $s_t^{(n)} \sim P_{\text{err}}$ when $t \in \mathcal{E}^{(n)}$ and $s_t^{(n)} \sim P_{\text{ord}}$ otherwise. Eqn. 1 characterizes the marginal composition of the pooled steps; it does not assume that steps within a trajectory are temporally independent.

Our method uses $\mathcal{D}_{\text{fail}}$ only to construct the reference representation space, without observing $\mathcal{E}^{(n)}$ or any responsible-agent annotations. Gold step-level labels are used only for *evaluation* and, when applicable, *hyperparameter selection* on a disjoint validation split.

3 SOAP: SPECTRAL SCORING WITH ATTENTION-GUIDED PROPAGATION

In this section, we introduce SOAP, a framework for localizing the decisive error from step-unlabeled failure trajectories. At a high level, SOAP consists of two components. *First*, we construct a spectral reference space from the step representations in $\mathcal{D}_{\text{fail}}$ and derive a base error score for each step (§3.1). *Second*, we estimate soft dependencies between steps from the attention patterns of the model and propagate downstream error evidence toward the earlier steps on which it depends (§3.2). The resulting score $\widetilde{S}(s_t)$ estimates how likely step s_t is to have introduced the failure.

3.1 SPECTRAL STEP SCORING FROM UNLABELED FAILURE TRAJECTORIES

The reference collection $\mathcal{D}_{\text{fail}}$ contains many ordinary steps but only a small number of decisive errors within each trajectory. Consequently, the representations from LLM $\mathcal{M}$ are dominated by recurring patterns of ordinary agent behavior, while decisive-error steps form a sparse, unlabeled component. Our key idea is to characterize this representation space through spectral decomposition and score each step according to its alignment with selected spectral directions.

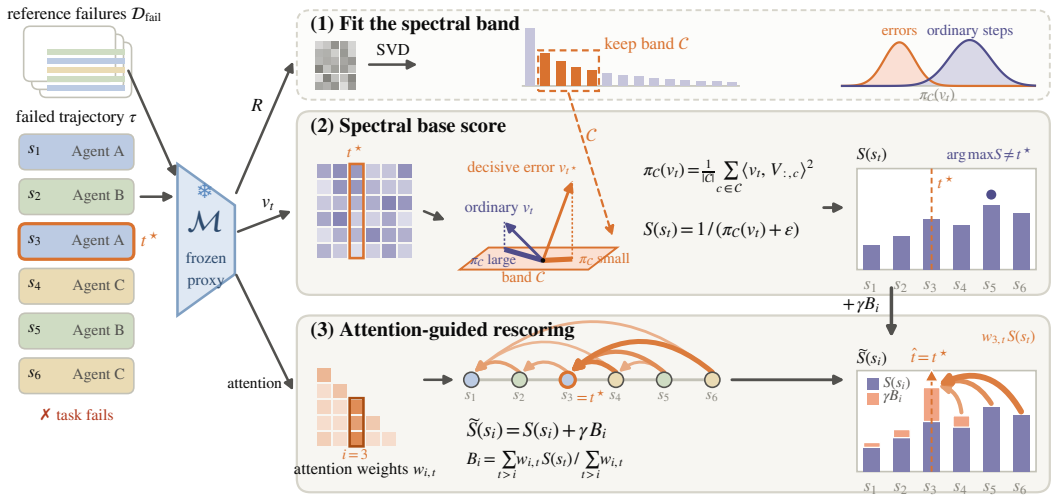


Figure 2: **Overview of SOAP.** [New layout for the main figure. Under refinement: too much text, need more compact, avoid overlapping components. —FINN] the fonts are not proper, please avoid purely relying AI for this.

For a trajectory $\tau = (s_1, \ldots, s_T)$, we encode each step in the context in which it was produced. Specifically, for step s_t, we provide the model $\mathcal{M}$ with the task description and the trajectory context $z_t = [x; s_1; \ldots; s_t]$. and then extract its d-dimensional step representation v_t. [refer details, e.g., context cutoff, in the appendix. —FINN]

Figure 2 summarizes the method.

Spectral reference space. We apply the same representation extraction procedure to every step in the reference collection $\mathcal{D}_{\text{fail}}$. Let $M = \sum_{n=1}^{N} T_n$ be the total number of reference steps. We stack their representations into $R = [(v_t^{(n)})^\top]_{n\in[N],\, t\in[T_n]} \in \mathbb{R}^{M\times d}$ where $v_t^{(n)}$ is representation of the t-th step in the n-th sequence or trajectory. We then compute its singular value decomposition

$$R = U\Sigma V^\top. \tag{2}$$

The columns of V describe orthogonal directions of variation among the unlabeled steps, ordered by decreasing singular value.

Importantly, the reference set R is dominated by ordinary steps because each failure trajectory typically contains *only a small number of decisive errors*. Consequently, the leading singular directions may primarily encode frequent shared structure, such as response style, trajectory format, and common tool-use patterns. Conversely, components at the extreme tail may contain irrelevant variation. We therefore do not assume that the most informative signal must lie in the leading components. Instead, we consider a contiguous spectral band to represent the decisive-error information $\mathcal{C} = \{c_{\text{begin}}, \ldots, c_{\text{end}} - 1\}$, whose boundaries are selected as described in §4.

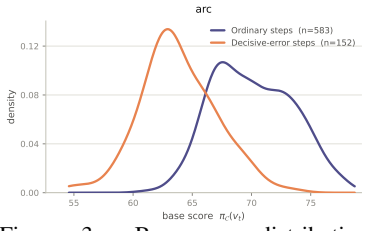


Figure 3: Base-score distribution on a CORRECT-ERROR test split. Decisive-error steps concentrate at smaller projection magnitudes than ordinary steps.

Spectral base score. For a step representation v_t, we measure its average squared projection onto the selected spectral band:

$$\pi_{\mathcal{C}}(v_t) = \frac{1}{|\mathcal{C}|}\sum_{c\in\mathcal{C}} \langle v_t, V_{:,c}\rangle^2. \tag{3}$$

Because ordinary steps dominate the unlabeled reference corpus, decisive-error steps exhibit smaller projection magnitudes on the selected directions than ordinary steps (Figure 3). We therefore define $S(s_t) = \frac{1}{\pi_{\mathcal{C}}(v_t)+\epsilon}$. so that a larger score indicates stronger decisive-error evidence. $\epsilon > 0$ is a small constant that ensures numerical stability. Note that R is constructed solely from the step-unlabeled reference collection where *no decisive-error or responsible-agent annotation is used*.

3.2 DEPENDENCY-AWARE RESCORING

Attention-guided step dependencies. [add context cutoff explanation —FINN]appendix The base score in Eqn. 3 evaluates each step independently and therefore cannot distinguish an error source from its downstream consequences. Once a decisive error occurs, later steps may inherit the corrupted context and exhibit stronger failure signals than the original error. An earlier step is therefore more plausible as the root cause when subsequent anomalous steps depend strongly on it. To capture this structure, we construct a soft dependency graph over the target trajectory.

For every pair (i, t) with $i < t$, we define a non-negative coefficient $w_{i,t}$ that measures how strongly the later step s_t depends on the earlier step s_i. We derive this coefficient from the post-softmax attention produced by $\mathcal{M}$. Specifically, we aggregate the attention routed from tokens in s_t to tokens in s_i, yielding an attention mass $m_{i,t}$. We then normalize over all predecessor steps:

$$w_{i,t} = \frac{m_{i,t}}{\sum_{j<t} m_{j,t}}, \qquad i < t. \tag{4}$$

Thus, $\sum_{i<t} w_{i,t} = 1$, and $w_{i,t}$ represents the relative dependency of step s_t on predecessor s_i. The resulting weights provide soft evidence about which preceding steps contribute to the proxy model's representation of a later step. They require neither repeated LLM prompting, which introduces substantial inference cost (Zhang et al., 2025c; Yeh et al., 2026), nor explicit dependency reconstruction through LLM-based graph reasoning (Rafi et al., 2026; Wang et al., 2026), nor task-specific post-training on attributed failure trajectories (Zhang et al., 2025a;b).

Rescoring. Given the dependency weights, we treat the error signal of a downstream step as additional evidence for the earlier steps on which it depends. For each step s_i, we compute the dependency-weighted average of the base scores of its downstream dependents:

$$B_i = \frac{\sum_{t>i} w_{i,t}\, S(s_t)}{\sum_{t>i} w_{i,t}}, \qquad B_T = 0. \tag{5}$$

We then combine the step's local error signal with the downstream evidence attributed to it:

$$\widetilde{S}(s_i) = S(s_i) + \gamma B_i, \qquad \gamma \in [0, 1]. \tag{6}$$

Here, γ controls the strength of dependency-aware rescoring, and $\gamma = 0$ recovers the base score.

The normalization in Eqn. 5 produces a dependency-weighted average, which prevents an early or frequently attended step from receiving a large correction merely because it has many downstream steps. Instead, a step is promoted when the later steps that depend on it consistently exhibit strong failure signals. Conversely, an anomalous downstream step contributes little evidence to a predecessor on which it places little dependency weight.

Finally, we predict the decisive-error step and the corresponding responsible agent as

$$\hat{t} = \arg\max_{i \in \{1,\ldots,T\}} \widetilde{S}(s_i), \qquad \hat{a} = a_{\hat{t}}. \tag{7}$$

4 EXPERIMENTS

4.1 SETUP

Datasets. We evaluate our method SOAP on 3 failure-attribution benchmarks comprising 5 subsets: WHO&WHEN (Zhang et al., 2025c), with algorithm-generated (WW-AG, 126 trajectories) and hand-crafted (WW-HC, 58 trajectories) subsets; CORRECT-ERROR (Yu et al., 2025) (CE, 2,226 trajectories); and TRACEELEPHANT (Chen et al., 2026), with Captain-Agent (TE-CAP, 85 trajectories) and Magentic-One (TE-MAG, 91 trajectories) subsets. They cover trajectories of varying lengths and diverse QA, coding, and tool-use tasks. Each trajectory is annotated with the decisive-error step and responsible agent. For each subset, we randomly split trajectories into unlabeled-reference, validation, and test sets using a $30/20/50\%$ ratio. The reference annotations are hidden and used only to construct the spectral matrix R; validation labels are used for hyperparameter selection, and test labels only for final evaluation. Splitting is performed at the trajectory level. Full statistics and preprocessing details are provided in Appendix A.1. [add number of steps,appendix —FINN]

Table 1: **Main comparison in the *without-ground-truth* setting.** Results are step-level accuracy (%), averaged over three seeds (std in Appendix C.1). No method receives the gold task answer. GPT denotes GPT-4o; results with GPT-5 as the judge are in Appendix C.3. **Supervised** marks methods that consume step-level error annotations. Best per column within each block in **bold**, second best underlined. [revised it —JUNLIN] [Added ErrorProbe, consider remove RAFFLES. Should remove supervised? —FINN]

Method	Supervised	WW-AG	WW-HC	CE	TE-Cap	TE-Mag
GPT						
Prompt-based methods						
All-at-Once (Zhang et al., 2025c)		14.81	6.90	28.39	16.28	5.80
Step-by-Step (Zhang et al., 2025c)		20.63	13.79	44.47	13.95	13.04
Binary Search (Zhang et al., 2025c)		22.22	4.60	9.70	9.30	6.52
CORRECT (Yu et al., 2025)	✓	15.34	4.60	35.12	10.85	13.77
CHIEF (Wang et al., 2026)		25.93	14.94	38.56	13.95	8.70
RAFFLES (Zhu et al., 2026a)		35.98	18.39	53.05	23.26	23.91
ErrorProbe (Li et al., 2026)		–	–	–	–	–
Qwen3.5-9B						
Prompt-based methods						
All-at-Once (Zhang et al., 2025c)		17.46	11.49	33.47	4.65	1.45
Step-by-Step (Zhang et al., 2025c)		10.58	16.09	31.15	14.73	10.87
Binary Search (Zhang et al., 2025c)		2.65	13.79	59.92	0.78	5.07
CORRECT (Yu et al., 2025)	✓	29.10	3.45	36.35	7.75	2.90
CHIEF (Wang et al., 2026)		28.57	3.45	29.39	3.88	2.17
RAFFLES (Zhu et al., 2026a)		46.03	27.59	**73.14**	29.46	23.91
ErrorProbe (Li et al., 2026)		39.68	21.84	55.03	23.26	20.29
Training-based methods						
StepFinder (Zhu et al., 2026b)	✓	15.87	13.33	30.03	18.29	6.67
OAT (Yeh et al., 2026)	✗	16.72	10.11	56.50	15.35	18.84
SOAP (ours)	✗	**47.62**	**34.48**	61.78	**35.66**	23.19
DeepSeek-R1-Distill-Llama-8B						
Prompt-based methods						
All-at-Once (Zhang et al., 2025c)		7.41	5.75	21.95	4.65	2.90
Step-by-Step (Zhang et al., 2025c)		16.93	9.20	9.04	11.63	4.35
Binary Search (Zhang et al., 2025c)		7.94	12.64	34.15	10.08	13.77
CORRECT (Yu et al., 2025)	✓	20.11	11.49	32.82	3.88	0.72
CHIEF (Wang et al., 2026)		17.46	2.30	8.40	7.75	9.42
RAFFLES (Zhu et al., 2026a)		28.04	8.05	35.84	20.16	10.87
ErrorProbe (Li et al., 2026)		24.34	10.34	35.97	17.05	8.70
Training-based methods						
StepFinder (Zhu et al., 2026b)	✓	18.94	10.80	36.91	14.73	4.20
OAT (Yeh et al., 2026)	✗	12.70	15.86	52.50	17.98	13.04
SOAP (ours)	✗	**45.50**	**28.74**	**64.68**	**42.64**	**30.43**

[**Baselines.** We compare SOAP against two families of baselines. First, *prompt-based* methods include All-at-Once (), Step-by-Step (), and Binary Search (Zhang et al., 2025c), the schema-retrieval method CORRECT (Yu et al., 2025), the causal-graph method CHIEF (Wang et al., 2026), the fault-reasoning judge RAFFLES (Zhu et al., 2026a), and the analyzer–verifier diagnoser ErrorProbe (Li et al., 2026). These methods ask a judge LLM to name the decisive step. Second, *training-based* methods include OAT (Yeh et al., 2026), which learns successful-trajectory dynamics through one-class training, and StepFinder (Zhu et al., 2026b), which trains a step classifier on synthetically corrupted trajectories with step-level error labels. —JUNLIN]

[**Implementation details.** We use Qwen3.5-9B (Qwen Team, 2025) and DeepSeek-R1-Distill-Llama-8B (Guo et al., 2025) as backbones. We obtain step representations by mean-pooling token hidden states. We select the representation layer, the spectral band $\mathcal{C}$, the attention layer band, and the propagation strength γ on the validation set (we show the stability of SOAP to each hyperparameter in Section 4.3). All training-based baselines use the same two backbones and test data as SOAP, with the hyperparameters recommended in their papers. Following Yeh et al. (2026), we additionally compare with prompt-based baselines with GPT-4o as the judge (results with GPT-5 are in Appendix C.3). Following Zhang et al. (2025c), we report *Step-Level Accuracy* (Additional metrics are in Appendix C.1). Further implementation details are provided in Appendices A.3 and A.4. add appendix for agent-level and other metrics —JUNLIN]

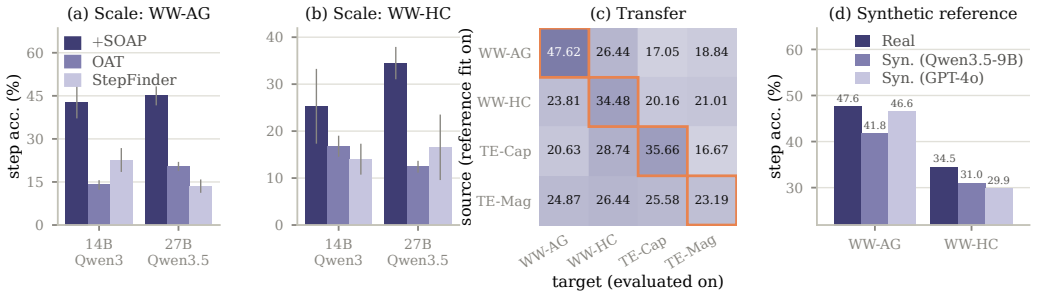


Figure 4: **(a) Scalability to larger backbones** on WW-AG. **(b) Scalability to larger backbones** on WW-HC. Step-level accuracy (%) with Qwen3-14B and Qwen3.5-27B as the backbone; error bars ±1 std. **(c) Generalization across data distributions:** source rows, target columns; the outlined diagonal is the in-distribution result of Table 1. **(d) Results with synthetic trajectories:** the reference is constructed on generated trajectories instead of the real train split. (c) and (d) use backbone Qwen3.5-9B.

4.2 Main Results

[**Comparison with competitive baselines.** Table 1 compares SOAP against both baseline families on the five subsets, strictly without the ground-truth answer. SOAP achieves the best step-level accuracy on all five subsets with both backbones, and outperforms the strongest prompt-based and training-based baselines by large margins (e.g., 47.62 vs. 35.98 for RAFFLES (Zhu et al., 2026a) and 16.72 for OAT (Yeh et al., 2026) on WW-AG). The advantage over the closed-source judges holds on both GPT-4o and GPT-5 (GPT-5 results in Appendix C.3). Notably, SOAP is the only method constructed on the actual trajectories while requiring no step-level annotation. The step-unlabeled failure trajectories already carry the attribution signal, which success-only training (OAT) and synthetic supervision (StepFinder) miss. Agent-level results for the same grid appear in Appendix C.1. —JUNLIN]

[**Results for the with-GT setting.** To assess the scalability of SOAP to the with-GT setting, we provide the gold task answer to all methods and compare them in Table 2. When provided with the ground-truth answer, SOAP remains the best method on WW-HC (34.48 vs. 28.74 for RAFFLES (Zhu et al., 2026a)) and the second best on WW-AG, where only RAFFLES surpasses it (46.56 vs. 43.39), while every training-based baseline trails by more than 20 points (e.g., 22.12 for OAT (Yeh et al., 2026) on WW-AG). The answer helps the judges more than it helps SOAP. Results on DeepSeek-8B appear in Appendix A.6. —JUNLIN] [table refilled 2026-09-05 with the Qwen3.5-9B judge; RAFFLES now leads WW-AG. —FINN]

Table 2: **Comparison in the *with-ground-truth* setting.** Results are step-level accuracy (%) on Who&When, with the gold task answer provided. All methods use Qwen3.5-9B as the backbone (DeepSeek-8B in Appendix A.6). Marks follow Table 1. [The table was revised by. —JUNLIN]

Method	Sup.	WW-AG	WW-HC
Prompt-based methods			
All-at-Once		21.16	11.49
Step-by-Step		17.46	18.39
Binary Search		2.12	13.79
CORRECT	✓	32.28	4.60
CHIEF		30.69	4.60
RAFFLES		**46.56**	28.74
ErrorProbe		40.74	19.54
Training-based methods			
StepFinder	✓	18.52	12.87
OAT	✗	22.12	8.05
SOAP (ours)	✗	43.39	**34.48**

[**Scalability to larger backbones.** To assess scalability, we evaluate SOAP on two larger backbones, Qwen3-14B and Qwen3.5-27B. As shown in Figure 4, SOAP achieves stable accuracy on both backbones and outperforms the SOTA baselines (e.g., 44.97 vs. 22.65 for StepFinder (Zhu et al., 2026b) on WW-AG). These results confirm that SOAP remains effective on larger backbones. —JUNLIN]

[**Generalization across data distributions.** We construct the reference on a source subset and directly evaluate SOAP on a different target subset, for every pair among {WW-AG, WW-HC, TE-Cap, TE-Mag}. As shown in Figure 4(c), SOAP transfers across subsets: for example, constructing on TE-Cap and testing on WW-HC achieves 28.74, compared with the in-domain result of 34.48. These results indicate that the spectral reference captures failure structure that is largely invariant to the source distribution. Results on DeepSeek-8B appear in Appendix B. —JUNLIN]

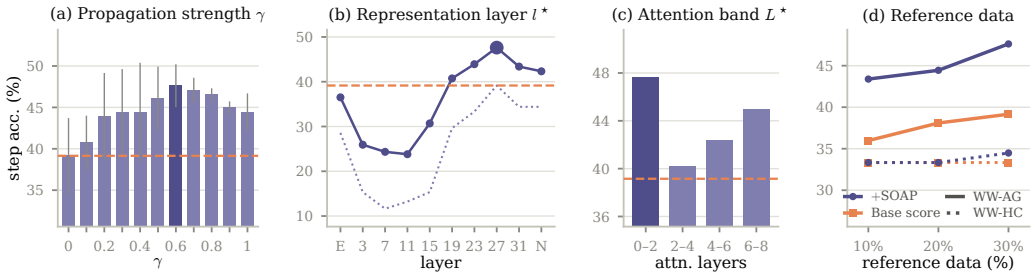


Figure 5: **Ablation studies.** (a) Sensitivity to the propagation strength γ, (b) sensitivity to the representation layer $l^\star$, (c) sensitivity to the attention layer band $L^\star$, and (d) quantity of unlabeled reference data. All results use backbone Qwen3.5-9B. Panels (a–c) vary one axis at a time from the selected configuration of Table 1 on WW-AG: the dark bar or large marker is the selected value, the dashed orange line is the base score with no rescoring (γ=0), and the dotted line in (b) is the base score at each layer. Panel (d) constructs the reference on $\{10, 20, 30\}\%$ of the corpus: SOAP in purple, the base score in orange; WW-AG solid, WW-HC dotted.

[**Results with synthetic trajectories.** SOAP constructs the reference on unlabeled trajectories, so we further evaluate whether it requires trajectories from the target system. We generate synthetic trajectories with Qwen3.5-9B and GPT-4o, construct R on them instead of the real train split, and keep the validation and test splits of Table 1 unchanged. As shown in Figure 4(d), the synthetic references stay close to the real one: for example, the GPT-4o-generated reference achieves 46.56 on WW-AG, compared with 47.62 for the real train split. These results indicate that SOAP does not require real trajectories from the target system. Generation details are in Appendix A.8. —JUNLIN]

4.3 ANALYSIS

[**Comparison with alternative scoring functions.** We compare the spectral base score with seven alternatives in Table 3: perplexity, random subspace, top subspace, tail subspace, full spectrum, and the ℓ_1 and ℓ_2 norms (details in Appendix A.3). Our spectral band remains the best on both subsets, and the best alternative reaches only 18.52 on WW-AG, compared with 39.15 for ours. These results indicate that the decisive-error signal lies in a selected spectral band rather than in likelihood or vector magnitude. —JUNLIN]

[**Importance of attention-guided rescoring.** We compare the attention-guided weights of Eqn. 5 with two content-agnostic weights (uniform, normalized or unnormalized) and two position heuristics (temporal bias, z-scored or raw, and earliest of top-5) in Table 4. Attention-guided rescoring matches or outperforms every alternative on all five subsets: for example, it reaches 47.62 on WW-AG, while the normalized uniform weights reach 40.21 and the unnormalized ones collapse to 16.93. These results indicate that the gain comes from which successors the attention selects, not from aggregation or position. —JUNLIN] this is not clear. we need extended explanation on each compared scoring.

Table 3: **Comparison with alternative scoring functions.** Results are step-level accuracy (%) on backbone Qwen3.5-9B, with no rescoring.

Scoring function	WW-AG	WW-HC
Perplexity	3.70	8.05
Random subspace	13.23	9.20
Top subspace	12.70	**33.33**
Tail subspace	6.35	6.90
Full spectrum	12.17	31.03
Norm ℓ_1	18.52	18.39
Norm ℓ_2	14.29	26.44
Spectral band (ours)	**39.15**	**33.33**

[**Sensitivity to the hyperparameters.** We vary the propagation strength γ, the representation layer $l^\star$, and the attention layer band $L^\star$ one at a time. As shown in Figure 5, accuracy peaks at $\gamma = 0.6$ and stays above the base score over a wide range, the signal is strongest at late layers, and the attention band changes accuracy the least. These results suggest that SOAP is stable to each choice. WW-HC, TraceElephant, DeepSeek-8B, and the context window w of the dependency weights are in Appendix B. —JUNLIN]

[**Quantity of unlabeled reference data.** We construct R on random subsamples covering $\{10, 20, 30\}\%$ of the corpus, with the validation and test splits fixed, and re-select the configuration at each fraction. As shown in

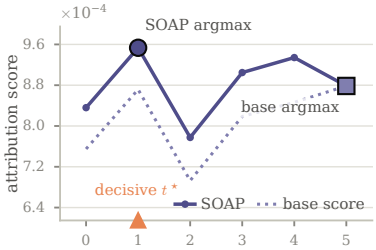


Table 4: **Alternatives to attention-guided rescoring.** Step-level accuracy (%), Qwen3.5-9B; all rows share the spectral base score. Weight rows replace only the dependency weights; position rows replace the whole correction (λ selected per column from $\{0.1, \ldots, 1.0\}$). The last row equals SOAP in Table 1. Best in **bold**. can we extend this table to the page rim [fixed]? also clarify what is z-scored?

Rescoring design	WW-AG	WW-HC	CE	TE-Cap	TE-Mag
Base score (no rescoring)	39.15	33.33	61.38	33.33	21.01
Content-agnostic weights					
Uniform, unnormalized	16.93	16.09	57.85	14.73	5.07
Uniform, normalized	40.21	**34.48**	60.78	**35.66**	15.94
Position heuristics					
Temporal bias, z-scored	40.74	**34.48**	61.62	34.11	21.01
Temporal bias, raw	22.22	3.45	41.65	6.20	5.07
Earliest of top-5	21.16	14.94	4.51	9.30	5.07
Attention-guided (ours)	**47.62**	**34.48**	**61.78**	**35.66**	**23.19**

Figure 5(d), SOAP needs little reference data: with a third of the train split (twelve trajectories on WW-AG and six on WW-HC), accuracy is within a few points of the full-data result (43.39 vs. 47.62 on WW-AG). These results indicate that a handful of unlabeled trajectories suffices to construct the reference. —JUNLIN]

Qualitative example. Figure 6 shows a TE-Cap trajectory on which rescoring flips the prediction from wrong to right. The base score peaks late, on an Orchestrator turn that only summarizes earlier steps. The dependency weights route this late anomaly back to the decisive error, a malformed program produced by the extraction agent four steps earlier.

Additional analysis. Due to space limitations, we defer additional experiments to the Appendix, including (1)

efficiency comparison to be in the appendix.

5 RELATED WORK

Agentic failure attribution has gained interest recently for ensuring LLM-based agents' reliability (Qi et al., 2026; Barke et al., 2026). Zhang et al. (2025c) formulate automated failure attribution, introduce the Who&When benchmark, and evaluate All-at-Once, Step-by-Step, and Binary Search LLM judges; complementary work categorizes common multi-agent failure modes (Cemri et al., 2026; Deshpande et al., 2025; Raj et al., 2026). Subsequent methods either improve inference-time prompting (Banerjee et al., 2025; Zhu et al., 2026a; Yu et al., 2025; Bakish et al., 2026) or train dedicated attributors. For instance, AgenTracer and StepFinder instead learn from labeled or synthetically corrupted trajectories using reinforcement learning or temporal-semantic modeling (Zhang et al., 2025a; Zhu et al., 2026b). These approaches, however, often rely on c*omputationally intensive inference or large-scale step-level error annotations*, limiting their practicality for real-world deployment. A recent approach OAT mitigates this by learning continuous latent dynamics from clean successful trajectories by one-class learning (Yeh et al., 2026), which *provides no decisive error signals* in their data unlike our adopted step-unlabeled failure trajectories thus underperforms our SOAP (Table **??**). Several methods model cross-step dependencies (Rafi et al., 2026). GraphTracer derives information-flow graphs from references to prior outputs and learns to trace root causes and propagation paths (Zhang et al., 2025b); CHIEF construct causal dependency graphs during prompting through multi-stage LLM reasoning (Wang et al., 2026). We compare with both of them in Table **??**. Liu et al. (2026) use attention signals for failure attribution but different from SOAP, they calculate attention scores on steps with high negative log-likelihood for two-stage localization.

[update the related work —FINN]

6 CONCLUSION

[We propose SOAP, a novel framework for failure attribution in LLM-based multi-agent systems. Our framework scores each step against a spectral reference constructed on unlabeled failure trajectories, and routes late anomalies back to the decisive error with attention-guided propagation. SOAP is lightweight, requires no step-level annotation, and can be readily applied to off-the-shelf backbones. Extensive experiments show that SOAP consistently outperforms prompt-based and training-based baselines across benchmarks, backbones, and settings. We hope our work can inspire future research on failure attribution and more broadly on building reliable multi-agent systems. —JUNLIN]

AI USE STATEMENT

(This section is **required** and does not count toward the page limit.)

In this work, we used generative AI tools for [tasks with required disclosure]. We have not used generative AI tools for [other tasks with required disclosure], and [the rest of the required disclosure tasks] are not applicable to this work. Additionally, we used generative AI tools for [tasks with recommended disclosure]. We have reviewed all AI-assisted work. [Elaborate. For example, "we checked LLM-generated research ideas for potential plagiarism through a manual literature survey", "LLM-generated code was verified and tested for correctness by 2 authors", etc.]. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

See the ICLR 2027 AI Policy for Authors for more details. This statement should not be more than 1 page.

ETHICS STATEMENT

(This section is **recommended** and does not count toward the page limit.)

If authors feel that their paper submission raises questions regarding the Code of Ethics, they are encouraged to include a paragraph of Ethics Statement (at the end of the main text before references) to address potential concerns where appropriate. Topics include, but are not limited to, studies that involve human subjects, practices to data set releases, potentially harmful insights, methodologies and applications, potential conflicts of interest and sponsorship, discrimination/bias/fairness concerns, privacy and security issues, legal compliance, and research integrity issues (e.g., IRB, documentation, research ethics). This statement should not be more than 1 page.

REPRODUCIBILITY STATEMENT

(This section is **recommended** and does not count toward the page limit.)

It is important that the work published in ICLR is reproducible. Authors are strongly encouraged to include a paragraph-long Reproducibility Statement at the end of the main text (before references) to discuss the efforts that have been made to ensure reproducibility. This paragraph should not itself describe details needed for reproducing the results, but rather reference the parts of the main paper, appendix, and supplemental materials that will help with reproducibility. For example, for novel models or algorithms, a link to an anonymous downloadable source code can be submitted as supplementary materials; for theoretical results, clear explanations of any assumptions and a complete proof of the claims can be included in the appendix; for any datasets used in the experiments, a complete description of the data processing steps can be provided in the supplementary materials. Each of the above are examples of things that can be referenced in the reproducibility statement.

REFERENCES

Yarden Bakish, Amir Dudai, Roy Ganz, Oren Nuriel, Elad Ben Avraham, Mor Shpigel Nacson, and Ron Litman. Adaptive influence graphs for failure attribution in multi-agent systems. *arXiv preprint arXiv:2608.24361*, 2026.

Adi Banerjee et al. Where did it all go wrong? a hierarchical look into multi-agent error attribution. *arXiv preprint arXiv:2510.04886*, 2025.

Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. AgentRx: Diagnosing AI agent failures from execution trajectories. *arXiv preprint arXiv:2602.02475*, 2026.

Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. URL https://openreview.net/forum?id=fAjbYBmonr.

Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 19888–19905, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.912. URL https://aclanthology.org/2026.acl-long.912/.

Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. Trail: Trace reasoning and agentic issue localization. *arXiv preprint arXiv:2505.08638*, 2025.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.

Jiazheng Li, Emine Yilmaz, Bei Chen, and Thu Le. Towards self-improving error diagnosis in multi-agent systems. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 2063–2077, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.98. URL https://aclanthology.org/2026.findings-acl.98/.

Yang Liu, Hongjiang Feng, Junsong Pu, and Zhuangbin Chen. MASPrism: Lightweight failure attribution for multi-agent systems using prefill-stage signals. *arXiv preprint arXiv:2605.07509*, 2026.

Yunjia Qi, Zehua Yin, Xintong Shi, Hao Peng, Songyuanyi Lu, Yixian Liu, Richeng Xuan, Yuhong Liu, Zhichao Hu, Xiaozhi Wang, Lei Hou, Bin Xu, and Juanzi Li. TRAJDEBUG: Tracing error lifecycle to identify critical failures in long-horizon agent trajectories. *arXiv preprint arXiv:2608.06346*, 2026.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Md Nakhla Rafi, Md Ahasanuzzaman, Dong Jae Kim, Zhijie Wang, and Tse-Hsun Chen. Falat: Tracing failures in llm agent trajectories via dependency-guided search, 2026. URL https://arxiv.org/abs/2606.00765.

Harsh Raj, Vipul Gupta, Anas Mahmoud, Razvan-Gabriel Dumitru, Darvin Yi, Aakash Sabharwal, and Yunzhong He. Model or harness? an interaction-centric taxonomy for localizing agent failures. *arXiv preprint arXiv:2607.28802*, 2026.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. doi: 10.1007/s11704-024-40231-1.

Yawen Wang, Wenjie Wu, Junjie Wang, and Qing Wang. From flat logs to causal graphs: Hierarchical failure attribution for llm-based multi-agent systems, 2026. URL https://arxiv.org/abs/2602.23701.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Samuel Yeh, Yiwen Zhu, Shaleen Deep, and Sharon Li. Tracing agentic failure from the flow of success. *arXiv preprint arXiv:2607.12747*, 2026.

Yifan Yu, Moyan Li, Shaoyuan Xu, Jinmiao Fu, Xinhai Hou, Fan Lai, and Bryan Wang. CORRECT: COndensed eRror RECognition via knowledge transfer in multi-agent systems. *arXiv preprint arXiv:2509.24088*, 2025.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer: Who is inducing failure in the llm agentic systems? *arXiv preprint arXiv:2509.03312*, 2025a.

Heng Zhang, Yuling Shi, Xiaodong Gu, Haochen You, Zijian Zhang, Lubin Gan, Yilei Yuan, and Jin Huang. Graphtracer: Graph-guided failure tracing in llm agents for robust multi-turn deep search, 2025b. URL https://arxiv.org/abs/2510.10581.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. In *International Conference on Machine Learning (ICML)*, 2025c.

Chenyang Zhu, Spencer Hong, Jingyu Wu, Kushal Chawla, Yuhui Tang, Youbing Yin, Nathan Wolfe, Erin Babinsky, and Daben Liu. RAFFLES: Reasoning-based attribution of faults for LLM systems. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7659–7688, Rabat, Morocco, March 2026a. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.359. URL https://aclanthology.org/2026.eacl-long.359/.

Taiyu Zhu, Yifan Wu, Weilin Jin, Ying Li, and Gang Huang. StepFinder: A temporal semantic framework for failure attribution in multi-agent systems. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2026b.

A Additional experimental details

A.1 Datasets and statistics

We evaluate on three failure-attribution benchmarks that together provide five reported subsets. Every trajectory is a failed run annotated with one decisive-error step and its responsible agent; we read both labels as they are released and index steps from zero.

Who&When. We use the two subsets of Zhang et al. (2025c): WW-AG holds 126 trajectories generated by CaptainAgent, and WW-HC holds 58 trajectories of Magentic-One whose decisive step was annotated by hand. The hand-crafted trajectories open with the user's question as a turn of its own; that turn is never a candidate step, so it is excluded from scoring by every method.

CORRECT-Error. The corpus of Yu et al. (2025) spans seven question pools (ARC, GAIA, HotpotQA, MATH-500, MMLU-Pro, MuSiQue, and 2WikiMultihopQA), 2,226 trajectories in total. We treat each pool as a subset with its own hyperparameter selection and report the column CE as the unweighted mean over the seven per-subset accuracies, so a small pool such as GAIA (50 trajectories) weighs as much as 2WikiMultihopQA (733).

TraceElephant. From Chen et al. (2026) we use the CaptainAgent subset TE-Cap (85 trajectories) and the Magentic-One subset TE-Mag (91 trajectories).

Splits and seeds. [remove this paragraph. —FINN] Every subset is split at the trajectory level into reference, validation, and test sets with a $30/20/50\%$ ratio: a seeded shuffle first separates the test half, and a second shuffle with the same seed separates reference from validation. A reported number is the mean over three consecutive seeds, the subset's *seed triple*, frozen once and shared by both backbones (Table 5). CORRECT-Error's seven subsets share one triple. The with-GT setting carries its own triples because it was frozen separately; the two settings therefore compare methods on the same protocol but not always on the same partition.

Table 5: The frozen seed triples. A triple names the three consecutive split seeds a subset is evaluated on; both backbones use the same triple.

Setting	WW-AG	WW-HC	CE (all seven)	TE-Cap	TE-Mag
Without GT	3, 4, 5	13, 14, 15	17, 18, 19	22, 23, 24	15, 16, 17
With GT	38, 39, 40	13, 14, 15	17, 18, 19	2, 3, 4	9, 10, 11

Trajectory statistics. Table 6 reports the size and shape of every subset. Trajectory length varies by an order of magnitude across subsets: [should only analyze length of WW-AG and WWW-HC for example. —FINN]

Table 6: Corpus statistics per subset: number of trajectories, turns per trajectory (mean, median, maximum), distinct agents per trajectory, the mean position of the decisive error (0-indexed) and its mean relative position in the trajectory, and the number of trajectories whose decisive error is the first turn. The CE row is the unweighted mean over its seven pools.

Subset	Agent system	Traj.	Turns			Agents	Decisive step		First
			mean	med.	max		mean	rel.	
WW-AG	CaptainAgent	126	8.7	10	10	3.6	3.0	0.37	20
WW-HC	Magentic-One	58	51.6	32	130	5.7	15.2	0.46	0
TE-Cap	CaptainAgent	85	20.5	13	59	2.7	6.0	0.42	6
TE-Mag	Magentic-One	91	29.3	28	50	2.1	11.4	0.45	6
CE-ARC	CORRECT	304	5.3	2	22	2.3	2.0	0.86	11
CE-GAIA	CORRECT	50	10.9	10	22	2.9	7.7	0.86	0
CE-HotpotQA	CORRECT	578	12.8	10	22	3.0	5.2	0.63	8
CE-MATH-500	CORRECT	157	6.3	5	21	3.1	3.3	0.83	3
CE-MMLU-Pro	CORRECT	92	6.1	2	22	2.6	2.0	0.79	6
CE-MuSiQue	CORRECT	312	14.7	21	22	3.0	6.1	0.59	6
CE-2WikiMQA	CORRECT	733	14.5	14	22	3.0	6.7	0.64	9
CE (mean)	CORRECT	2,226	10.1	9.1	22	2.8	4.7	0.74	43

A.2 Input format and context truncation

Serialization. To score step s_t, the proxy $\mathcal{M}$ reads one user turn of its own chat template that contains the task description followed by turns $s_1, \ldots, s_t$, each serialized as `[{agent}] - Step {i}: {content}` and separated by a blank line, and then the template's generation prompt, so the proxy reads the trajectory in the state of a model about to respond. No system prompt is sent. Each turn is tokenized on its own and the token ids are concatenated, so the token span of the scored step is known exactly and never shifts with the template; reasoning-mode scaffolding is disabled (Qwen3.5's thinking switch is off, and the empty `<think>` block that DeepSeek-R1-Distill's template opens is closed immediately). The scored step carries no end-of-sequence token.

Context truncation. The context budget is 8,192 tokens for every subset and backbone. When the task description, the preceding turns, and the scored step exceed it, we drop the oldest turn and re-assemble the input, repeating until it fits; the task description is the first thing dropped, so long trajectories (WW-HC reaches 130 turns) are scored without it. In the with-GT setting the gold-answer block is pinned and survives every drop (Appendix A.6). A step whose own content exceeds the budget keeps only its last 8,192 tokens; this happens on no trajectory of the reported subsets.

A.3 Implementation details

Step representations. v_t is the mean of the hidden states over the scored step's own tokens at one layer, averaged in single precision and stored in half precision. The proxy runs in `bfloat16`, one step per forward pass, without padding or key-value caching. For Qwen3.5-9B, a hybrid model whose stack interleaves three linear-attention blocks with each full-attention block, we store the output of the eight full-attention blocks (3, 7, …, 31), the embedding layer, and the last block after the final norm; for DeepSeek-R1-Distill-Llama-8B we store all 32 blocks plus the same two. The representation layer $l^\star$ is selected over these positions together with an ensemble position that averages the z-scored scores of the middle third of the stored blocks; the ensemble is selected in 4 of the 44 reported cells, all on CORRECT-Error.

Attention mass. $m_{i,t}$ is computed inside the same forward pass without materializing an attention matrix over the whole context: the scored step's query vectors are scored against every key, the causal mask is re-applied, the softmax is taken in single precision, the probabilities are averaged over the step's query tokens and over heads, and the mass landing in each predecessor step is summed over that step's key tokens. Mass that lands on template tokens, on the step itself, or on attention sinks is discarded by the renormalization of Eqn. 4. The attention layer band $L^\star$ averages $m_{i,t}$ over one of four equal bands of the attention blocks: blocks $\{0\text{–}2, 2\text{–}4, 4\text{–}6, 6\text{–}8\}$ of Qwen3.5-9B's eight full-attention blocks and $\{0\text{–}8, 8\text{–}16, 16\text{–}24, 24\text{–}32\}$ of DeepSeek's 32.

Spectral base score. The reference matrix R stacks the representations of every step of the reference split, uncentered; we keep the 20 leading right-singular vectors and consider every contiguous band $[c_{\text{begin}}, c_{\text{end}})$ with $0 \le c_{\text{begin}} < c_{\text{end}} \le 20$, i.e., 210 bands. The score is $S(s_t) = 1/(\pi_{\mathcal{C}}(v_t) + \epsilon)$ with $\epsilon = 10^{-12}$. Ranking is within a trajectory and ties resolve to the earliest step.

Rescoring as implemented. The main text states the correction of Eqn. 5 over all predecessors. The implementation adds one sparsification: before propagation, each step s_t keeps only its w strongest predecessors under $w_{i,t}$ and renormalizes the kept weights to sum to one, with $w \in \{1, 2, 3, 4, 5, \text{all}\}$ selected per cell; $w = $ all recovers the main-text form. A predecessor that is never a candidate step (the human question turn of WW-HC, or the pinned answer block with GT) is removed after this selection. Blame then flows through the trimmed weights: $\widetilde{S}(s_i) = S(s_i) + \gamma\, B_i$ with B_i the w-weighted mean of the base scores of the successors that kept s_i, so a step that no successor selected passes through unchanged, and the last step, which has no successors, always does. All γ are evaluated in one pass over the original base scores; the $\gamma = 0$ rows reproduce the base score exactly and are asserted to on every run.

Hyperparameter grids. The base grid is the product of the stored positions (11 for Qwen3.5-9B, 35 for DeepSeek) and the 210 bands. The rescoring grid, expanded only for the selected base configuration, is the product of the four attention bands, $\gamma \in \{0, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0\}$, and the six windows w. Appendix A.5 gives the selection rule; Table 7 lists the selected configuration of every reported cell.

Table 7: The selected configuration of SOAP per cell: representation position (`act/`l is the output of block l, `normed` after the final norm, `embed` the embedding layer, `ens` the mid-stack ensemble), spectral band $\mathcal{C}$, attention band $L^\star$, propagation strength γ, and window w, with the resulting test step-level accuracy (%).

Setting	Backbone	Subset	Position	Band	$L^\star$	γ / w	Acc.
Without GT	Qwen3.5-9B	WW-AG	act/27	[1, 7)	0–2	0.6 / 1	47.62
		WW-HC	act/31	[0, 5)	4–6	0.1 / 2	34.48
		TE-Cap	act/23	[0, 3)	6–8	0.1 / 5	35.66
		TE-Mag	act/23	[0, 2)	6–8	1.0 / 4	23.19
	DeepSeek-8B	WW-AG	act/3	[1, 5)	24–32	1.0 / 1	45.50
		WW-HC	act/31	[0, 4)	–	0.0 / –	28.74
		TE-Cap	act/31 normed	[0, 18)	8–16	0.2 / all	42.64
		TE-Mag	act/10	[0, 4)	24–32	0.1 / 1	30.43
	Qwen3-14B	WW-AG	act/35	[1, 5)	0–10	0.4 / 3	42.86
		WW-HC	act/16	[0, 1)	30–40	0.1 / all	25.29
	Qwen3.5-27B	WW-AG	act/59	[1, 7)	0–4	0.4 / 1	44.97
		WW-HC	act/63 normed	[0, 4)	8–12	0.1 / all	34.48
With GT	Qwen3.5-9B	WW-AG	act/27	[1, 11)	0–2	1.0 / all	43.39
		WW-HC	act/31	[0, 5)	4–6	0.1 / 2	34.48
		TE-Cap	act/23	[0, 7)	2–4	0.2 / 2	36.43
		TE-Mag	embed	[1, 18)	6–8	0.8 / 4	21.01
	DeepSeek-8B	WW-AG	act/30	[1, 16)	0–8	0.8 / 3	44.97
		WW-HC	act/31	[0, 4)	–	0.0 / –	29.89
		TE-Cap	act/29	[0, 11)	24–32	0.1 / 2	42.64
		TE-Mag	ens	[0, 9)	24–32	0.4 / 2	36.96
Without GT	Qwen3.5-9B	CE-ARC	embed	[1, 3)	6–8	0.1 / 1	81.58
		CE-GAIA	ens	[1, 4)	6–8	0.2 / all	69.33
		CE-HotpotQA	act/15	[1, 3)	4–6	0.2 / 1	46.48
		CE-MATH-500	act/7	[0, 20)	–	0.0 / –	61.18
		CE-MMLU-Pro	act/11	[0, 3)	–	0.0 / –	79.71
		CE-MuSiQue	act/3	[1, 3)	–	0.0 / –	43.59
		CE-2WikiMQA	act/15	[1, 3)	6–8	0.1 / 1	50.59
	DeepSeek-8B	CE-ARC	act/13	[0, 3)	–	0.0 / –	87.72
		CE-GAIA	act/13	[0, 4)	–	0.0 / –	72.00
		CE-HotpotQA	act/16	[0, 4)	–	0.0 / –	52.71
		CE-MATH-500	act/15	[1, 3)	–	0.0 / –	59.92
		CE-MMLU-Pro	act/8	[0, 3)	–	0.0 / –	80.43
		CE-MuSiQue	act/13	[0, 6)	–	0.0 / –	44.66
		CE-2WikiMQA	act/12	[0, 3)	0–8	0.1 / 1	55.31

Alternative scoring functions. The seven alternatives of Table 3 share the selected layer and band width $|\mathcal{C}|$ of the spectral score and replace only the score. *Perplexity* is the mean negative log-likelihood of the step's tokens under the proxy in context, read as higher = error. *Random subspace* projects onto a random orthonormal basis of dimension $|\mathcal{C}|$, redrawn per seed; *top subspace* uses the band $[0, |\mathcal{C}|)$; *tail subspace* the last $|\mathcal{C}|$ of the 20 computed components; *full spectrum* all 20. The ℓ_1 and ℓ_2 norms score $\|v_t\|_1$ and $\|v_t\|_2$. Every projection and norm score is read as lower = error, the orientation of the spectral score; orientations are fixed, not selected. Where the selected band starts at component 0 the top subspace *is* the spectral band, which explains the ties in Table 3.

A.4 BASELINES

All baselines are evaluated on SOAP's test splits: for every cell, a method's predictions are scored on the same trajectories, with the same step and agent rules, and averaged over the same seed triple, so a prompting cell and a SOAP cell count the same trajectories. A prediction that is missing or cannot be parsed counts as an error and stays in the denominator. this might be too much. we need concise explanation on the method and the implementation details.

Prompt-based methods. All-at-Once, Step-by-Step, and Binary Search run the code released with Who&When (Zhang et al., 2025c); CORRECT (Yu et al., 2025) and CHIEF (Wang et al., 2026) are reproduced from their released code with prompts kept byte-identical, apart from one sentence

added to CHIEF's prompts that states that steps are indexed from zero, which the vendored data conveyed through a field our corpus lacks. RAFFLES (Zhu et al., 2026a) releases no code, so we implement its judge–evaluator loop from the paper: the prompts are transcribed from its appendix, the judge proposes a step, three evaluators and one rule-based consistency check score it, and the loop stops when the summed confidence exceeds the paper's threshold or after its iteration budget. The Who&When methods decode with the released settings (temperature 0.6, top-p 0.95, 1,024 new tokens, fixed seed); CORRECT's detection and RAFFLES decode greedily, as their papers prescribe. GPT-4o and GPT-5 are called through the API; Qwen3.5-9B and DeepSeek-R1-Distill-Llama-8B are served with vLLM in `bfloat16` with thinking disabled. The with-GT and without-GT runs differ only in one line of the prompt, which states the gold answer. For CHIEF and RAFFLES under GPT-5 we did not run the CORRECT-Error corpus, whose 2,226 trajectories and multi-call protocols made those cells the most expensive of the sweep; those cells are blank in Appendix C.3.

OAT. OAT learns the latent dynamics of *successful* trajectories and flags the steps of a failed trajectory that depart from them. Each trajectory is serialized as one document — the task description followed by every step under a header naming its index and agent — and encoded in a single forward pass of the backbone; a step's representation is the mean of its tokens' last-layer hidden states, reduced to 64 dimensions by PCA fitted on the training trajectories. A neural controlled differential equation is then trained to predict each step's latent state from the trajectory so far, with squared-error loss. Training is one-class: because our benchmarks contain only failures, OAT trains on the 103 successful tool-use trajectories released with the paper (MCP-Atlas), which is the paper's own out-of-distribution protocol; no trajectory from our benchmarks enters training, and one trained model serves every subset. At test time, CORAL aligns the second-order statistics of each subset's representations with those of the training set (using no labels); a step's anomaly score is the squared distance between its predicted and observed latent state, and the predicted decisive step is the argmax. OAT predicts no agent, so the responsible-agent prediction is the agent of the predicted step. Following the released code, human and environment turns are not scored; a trajectory whose annotated step falls on such a turn is kept and counted as an error. We train five models per backbone (seeds 42–46) and report the mean. In the with-GT setting, the gold answer is appended to the task description with the same phrasing the prompt-based baselines use; the training corpus carries no answers, so the trained model is shared between the two settings. We note that the paper's own benchmark annotates every step that contributed to a failure and reports set metrics, whereas our benchmarks annotate one decisive step, so the numbers printed in the paper are not directly comparable to Table 1.

StepFinder. StepFinder is a supervised step classifier. Each step is embedded twice — 128 dimensions for its text, 32 for the name of its agent — and the sequence of step embeddings passes through a BiLSTM, an agent-aware attention layer, and a scoring head with multi-scale differencing and a position prior; training minimizes a cross-entropy loss across the steps of each trajectory plus a self-supervised next-step prediction loss. Training uses the step-labeled failure corpora released with the paper, which regenerate each annotated task into roughly eighteen alternative failures: 1,564 trajectories in the CaptainAgent style and 2,604 in the Magentic-One style. Because the agent embedding is a text embedding of the agent's name, each evaluation subset is paired with the corpus from the agent system that produced it: WW-AG and TE-Cap with the CaptainAgent corpus, WW-HC and TE-Mag with the Magentic-One corpus. CE matches neither system and falls back to the Magentic-One corpus, so its column is out-of-domain transfer. To place StepFinder on the same backbones as SOAP, we replace the paper's encoder (Qwen3-Embedding-0.6B) with the backbone's last-layer hidden state at the final token and keep the leading 128 (text) and 32 (agent) coordinates, exactly as the released code slices its embeddings; the slice is principled for the paper's embedding model, whose leading coordinates are trained to carry the most information, and arbitrary for a decoder. For reference, with the paper's own encoder StepFinder reaches 24.02 / 11.72 / 32.75 / 17.98 / 21.16 on the five subsets of Table 1 — higher than the backbone rows on WW-AG and TE-Mag, comparable elsewhere, and still far below SOAP on every subset. Our setup departs from the released code in two places, both protocol repairs. First, the released training loop selects the checkpoint by accuracy on the test set; we instead hold out a fraction of the training tasks — split by question, so the near-duplicate regenerations of one task never straddle the boundary — and select on that holdout, never touching the test data. Second, we score one trajectory at a time, so the position prior is normalized by the trajectory's own length as in the paper's equations rather than by the padded width of a batch. As for OAT, we train five models per backbone (seeds 42–46), report the mean, and derive the responsible agent from the predicted step. In the with-GT setting, the gold answer is appended

to the content of the first step, since StepFinder encodes no separate task description; the trained models are shared between the two settings. Finally, the regenerated training corpus shares tasks — though not trajectories — with parts of our test data: 22 of 85 TE-Cap, 51 of 91 TE-Mag, and 30 of 50 CE-GAIA trajectories are built on tasks that also appear in training, while both Who&When subsets are disjoint. This overlap can only favor the baseline, so we leave it in place.

Evaluation. Both baselines write one prediction per trajectory; a trajectory that receives no usable prediction — for OAT, one whose annotated step is never scored — counts as an error. A reported cell is the mean over the three test splits of the subset's seed triple and, within each split, over the five training seeds, under the same scoring rules as every other method in Table 1.

A.5 SELECTION PROTOCOL

The rule. Within a frozen triple, the configuration of a cell is chosen in two stages by the same rule: over the base grid, pick the (position, band) with the highest mean test step-level accuracy over the three seeds, breaking ties by agent-level accuracy; then, over the rescoring grid expanded for that base configuration, pick ($L^\star$, γ, w) the same way. The configuration must exist in all three seeds. The seed triples themselves (Table 5) were chosen from a sweep over 48 candidate triples for Who&When and TraceElephant (18 for CORRECT-Error) by the sum of the two backbones' test accuracies; across candidate triples the reported number moves by up to 15 points, so the seeds matter more than the exact winner. Both stages read the test split. This is the protocol behind every number in Tables 1 and 2, and the ablations start from the configuration it selects; it is optimistic, and it is applied uniformly, so every method that carries a selectable knob (the λ of the position heuristics, the per-fraction configurations of the data-quantity study) is selected the same way.

Validation-selected counterpart. To measure the optimism, we re-select every cell on the validation split (mean validation step-level accuracy over the triple, same two stages, same tiebreak) and report its test accuracy. Table 8 shows both. Validation selection lowers every SOAP number, by 4 to 15 points on Who&When and TraceElephant and by under 6 on CORRECT-Error; the drop is largest where the validation split is smallest (twelve trajectories on WW-HC, eighteen on TE-Mag). The ordering that the paper argues survives it: under validation selection, rescoring still improves on the base score in 7 of the 10 without-GT cells and ties it elsewhere (e.g., 37.57 vs. 32.28 on WW-AG and 40.74 vs. 34.39 on DeepSeek WW-AG), and SOAP stays above OAT and StepFinder in every cell of Table 1. These results indicate that the gain from attention-guided rescoring is not an artifact of test selection, while the absolute accuracies of Table 1 should be read as the upper end of what a small validation split can recover.

Table 8: Test-selected vs. validation-selected configurations. Step-level accuracy (%) on the test split when the configuration is chosen on the test split (the protocol of Table 1) or on the validation split, for the base score and SOAP, both backbones and both settings.

Backbone	Method	Selected on	WW-AG	WW-HC	CE	TE-Cap	TE-Mag
Without GT							
Qwen3.5-9B	Base	test	39.15	33.33	61.38	33.33	21.01
		validation	32.28	26.44	59.47	19.38	8.70
	SOAP	test	47.62	34.48	61.78	35.66	23.19
		validation	37.57	25.29	59.35	22.48	8.70
DeepSeek-8B	Base	test	38.62	28.74	64.19	40.31	29.71
		validation	34.39	24.14	58.47	31.01	23.19
	SOAP	test	45.50	28.74	64.68	42.64	30.43
		validation	40.74	24.14	59.15	31.78	23.19
With GT							
Qwen3.5-9B	Base	test	41.27	33.33	60.30	32.56	15.94
		validation	38.10	28.74	58.41	19.38	5.07
	SOAP	test	43.39	34.48	60.59	36.43	21.01
		validation	39.68	24.14	57.92	22.48	5.80
DeepSeek-8B	Base	test	40.21	29.89	64.81	41.09	35.51
		validation	34.92	24.14	61.57	21.71	33.33
	SOAP	test	44.97	29.89	65.19	42.64	36.96
		validation	34.92	24.14	61.61	28.68	33.33

A.6 WITH-GROUND-TRUTH SETTING

How SOAP receives the answer. In the with-GT setting the proxy's context opens with a pinned block, "The problem is: {question}. The Answer for the problem is: {answer}", phrased as in the prompting baselines so the proxy reads the answer in the same register. The block precedes every turn, survives context truncation, is never pooled into a step representation, and is treated as a predecessor that attention may point to but that is never a candidate step and is removed before propagation (Appendix A.3). Nothing else changes: the same grids, the same selection rule, its own frozen triples (Table 5). OAT appends the answer to the task description and StepFinder to the first step, as described in Appendix A.4.

Full results. Table 9 extends Table 2 to all five subsets, the GPT-4o judge, and DeepSeek-R1-Distill-Llama-8B. The open-weight judges were run with the answer on all six prompt-based methods for the two Who&When subsets and on the three Who&When methods for TraceElephant; the remaining cells were not run and are blank. The picture of Table 1 holds: SOAP is best or second on every column, the answer helps the judges more than it helps SOAP (RAFFLES with GPT-4o gains 4 points on CE and 11 on TE-Mag; SOAP moves by under 5 points everywhere except DeepSeek TE-Mag, where it gains 6.5), and on DeepSeek SOAP keeps the best number in all five columns.

Table 9: Step-level accuracy (%) in the with-ground-truth setting on all five subsets. The prompt-based rows use the judge named in the block; the training-based rows and SOAP use the backbone named in the block. A blank cell was not run. The best result in each column within a block is highlighted in **bold**.

Method	WW-AG	WW-HC	CE	TE-Cap	TE-Mag
GPT-4o					
All-at-Once	15.87	3.45	24.50	9.30	6.52
Step-by-Step	22.75	18.39	59.29	18.60	19.57
Binary Search	28.57	4.60	18.44	9.30	6.52
CORRECT	15.87	4.60	37.09	10.85	10.14
CHIEF	24.34	11.49	42.10	14.73	13.77
RAFFLES	**42.33**	**27.59**	**68.41**	**27.91**	**28.26**
Qwen3.5-9B					
All-at-Once	21.16	11.49	–	6.98	1.45
Step-by-Step	17.46	18.39	–	24.81	21.01
Binary Search	2.12	13.79	–	0.00	10.14
CORRECT	32.28	4.60	–	–	–
CHIEF	30.69	4.60	–	–	–
RAFFLES	**46.56**	28.74	–	–	–
StepFinder	18.52	12.87	28.52	13.64	8.99
OAT	22.12	8.05	56.21	17.98	20.00
SOAP (w/o rescoring)	41.27	33.33	60.30	32.56	15.94
SOAP	43.39	**34.48**	**60.59**	**36.43**	**21.01**
DeepSeek-R1-Distill-Llama-8B					
All-at-Once	21.16	5.75	–	8.53	4.35
Step-by-Step	15.34	3.45	–	10.85	5.07
Binary Search	6.35	12.64	–	4.65	7.25
CORRECT	19.58	11.49	–	–	–
CHIEF	14.29	11.49	–	–	–
RAFFLES	32.80	13.79	–	–	–
StepFinder	18.94	9.89	24.99	15.04	6.23
OAT	20.85	17.01	52.39	18.14	13.77
SOAP (w/o rescoring)	40.21	29.89	64.81	41.09	35.51
SOAP	**44.97**	**29.89**	**65.19**	**42.64**	**36.96**

A.7 COMPUTE RESOURCES AND TIME

Software and hardware. All experiments use Python 3.10, PyTorch 2.13, and Transformers 5.15 on NVIDIA H200 GPUs with 141 GB memory, one GPU per run.

Extraction and scoring time. SOAP trains nothing and calls no judge: its only model cost is one forward pass per step to extract the representation and the attention mass, both from the same pass.

With Qwen3.5-9B this takes about 10 minutes for WW-AG and 2 hours 15 minutes for WW-HC, whose trajectories are five times longer; DeepSeek-8B is comparable. For the larger backbones of Appendix C.4, WW-HC takes 116 seconds per trajectory at 14B and 197 seconds at 27B. Once representations are stored, fitting the reference and scoring the full base grid of a cell takes 7–10 seconds per seed on one GPU, and rescoring evaluates all γ at once; selecting a new configuration or a new seed triple therefore needs no further forward passes. By contrast, the prompt-based baselines required 53,687 judge requests and 1.8×10^8 prompt tokens for the API sweep behind Tables 1 and 9, and RAFFLES issues up to five judge and evaluator calls per iteration per trajectory. OAT and StepFinder train small models on top of the same kind of representations and add no inference cost beyond the forward pass.

A.8 Synthetic reference trajectories

The synthetic references of Figure 4(d) are produced by running the agent system that generated each Who&When subset end to end, with an open or a closed model as the agents' backbone: CaptainAgent for WW-AG and Magentic-One for WW-HC, each with Qwen3.5-9B or GPT-4o as the language model. The systems run on the same question pools as the benchmarks (GAIA and AssistantBench), and whatever fails, fails on its own; no error is injected and no trajectory is edited. Each corpus is then filtered to the trajectories whose question appears in the target subset, so the reference covers the tasks a practitioner wants to diagnose: 126 of 198 generated trajectories for WW-AG with GPT-4o and 124 of 196 with Qwen3.5-9B (covering 126 and 124 of the 126 questions), and 55 of 198 and 55 of 1,502 for WW-HC (55 of 58 questions). The trajectories carry no step label and none is read; they are used, successful or not, only to construct R. The same questions appear in the reference and in the validation and test splits by design, since the use case is to re-run one's own agents on the tasks under diagnosis; the runner records the overlap per split. The validation and test splits, the seed triples, and the selection rule are those of Table 1; only the reference changes, and the configuration is re-selected per reference corpus. Full results for both backbones are in Appendix C.6.

B Additional ablation studies

The main text ablates SOAP on WW-AG and WW-HC with backbone Qwen3.5-9B (Section 4.3). This appendix repeats every ablation on the remaining cells, TE-Cap and TE-Mag on Qwen3.5-9B and all four subsets on DeepSeek-R1-Distill-Llama-8B, under the same protocol: start from the selected configuration of Table 7, vary one axis, hold everything else fixed. One caveat recurs: DeepSeek-8B's selected configuration on WW-HC has $\gamma = 0$, so rescoring is a no-op there and every row or bar that varies only the rescoring equals the base score by construction.

B.1 Scoring functions

Table 10 extends Table 3 to the remaining cells. The spectral band wins or ties every column. Where the selected band starts at component 0 (WW-HC, TE-Cap, and TE-Mag on both backbones) the top subspace *is* the selected band, so those ties hold by construction; the discriminating column is WW-AG, where the selected band starts at component 1 and dropping the leading direction is worth 24 points over the top subspace on DeepSeek-8B (38.62 vs. 14.29), as it is on Qwen3.5-9B. DeepSeek-8B's near-tie between the full spectrum and the band on TE-Cap (39.53 vs. 40.31) follows from its wide selected band $[0, 18)$, which nearly spans the computed spectrum. These results confirm that where the band sits matters, not only its width.

B.2 Rescoring designs

Table 11 is the DeepSeek-8B counterpart of Table 4. The picture repeats: the unnormalized sum collapses toward predicting the first step (16.40 on WW-AG), the normalized mean hovers around the base score, and no position heuristic reaches attention-guided rescoring where rescoring matters (45.50 vs. 38.62 for the best alternative on WW-AG). The WW-HC column is flat at the base score because that cell selected $\gamma = 0$; the same holds for six of the seven CE subsets on DeepSeek-8B, which is why the CE column moves little. These results indicate that the gain comes from which successors the attention selects, on this backbone as on Qwen3.5-9B.

Table 10: Alternative per-step scoring functions on the remaining cells. Step-level accuracy (%) with no rescoring; orientations fixed as in Table 3. The best result in each column is highlighted in **bold**.

	Qwen3.5-9B		DeepSeek-8B			
Scoring function	TE-Cap	TE-Mag	WW-AG	WW-HC	TE-Cap	TE-Mag
Perplexity	6.20	7.97	4.23	14.94	5.43	3.62
Random subspace	17.83	7.25	10.58	16.09	12.40	12.32
Top subspace	**33.33**	**21.01**	14.29	**28.74**	**40.31**	**29.71**
Tail subspace	6.20	1.45	10.58	4.60	18.60	5.07
Full spectrum	32.56	17.39	16.93	19.54	39.53	22.46
Norm ℓ_1	15.50	5.07	33.86	16.09	20.16	8.70
Norm ℓ_2	27.13	15.22	15.87	20.69	26.36	9.42
Spectral band (Ours)	**33.33**	**21.01**	**38.62**	**28.74**	**40.31**	**29.71**

Table 11: Alternatives to attention-guided rescoring on DeepSeek-8B. Step-level accuracy (%) on all five subsets; rows as in Table 4. The best result in each column is highlighted in **bold**.

Rescoring design	WW-AG	WW-HC	CE	TE-Cap	TE-Mag
Base score (no rescoring)	38.62	**28.74**	64.19	40.31	29.71
Content-agnostic weights					
Uniform, unnormalized	16.40	**28.74**	57.01	10.08	4.35
Uniform, normalized	35.45	**28.74**	62.14	41.86	28.99
Position heuristics					
Temporal bias, z-scored	38.10	**28.74**	63.33	41.09	28.26
Temporal bias, raw	38.62	3.45	42.05	6.20	20.29
Earliest of top-5	29.10	10.34	2.39	13.18	5.80
Attention-guided (Ours)	**45.50**	**28.74**	**64.68**	**42.64**	**30.43**

B.3 Context window of the dependency weights

We vary the window w, which controls how many predecessors each step distributes its blame over, with everything else at the selected configuration. As shown in Table 12, the preference is dataset-shaped. On Who&When both backbones favor the sharpest window: accuracy is highest at $w = 1$–2 and declines toward all predecessors (47.62 at $w = 1$ vs. 40.21 at all, Qwen3.5-9B WW-AG), so restricting each step to its few strongest dependencies keeps the correction targeted, while averaging over every predecessor dilutes it back toward the uniform weights of Table 4. On TE-Cap wider windows win on both backbones, up to all predecessors on DeepSeek-8B. A badly chosen w can land below the base score (Qwen3.5-9B TE-Mag at $w = 1$: 15.94 vs. 21.01). These results indicate that w is worth selecting rather than fixing.

Table 12: Context window w of the dependency weights. Step-level accuracy (%) when each step keeps its w strongest predecessors, everything else at the selected configuration; the selected w is highlighted in **bold**. DeepSeek-8B's selected γ on WW-HC is 0, so that row is constant by construction.

Backbone	Subset	$w = 1$	2	3	4	5	all
Qwen3.5-9B	WW-AG	**47.62**	41.80	40.74	40.21	40.21	40.21
	WW-HC	33.33	**34.48**	32.18	32.18	32.18	32.18
	TE-Cap	28.68	29.46	34.88	34.88	**35.66**	34.88
	TE-Mag	15.94	20.29	18.84	**23.19**	18.84	18.12
DeepSeek-8B	WW-AG	**45.50**	38.62	33.33	34.39	37.57	37.57
	WW-HC	28.74	28.74	28.74	28.74	28.74	**28.74**
	TE-Cap	35.66	38.76	41.09	41.09	41.86	**42.64**
	TE-Mag	**30.43**	30.43	30.43	29.71	28.99	28.99

B.4 Sensitivity to the hyperparameters on all cells

Figures 7 and 8 repeat Figure 5 on all eight (backbone, subset) cells and add the window w as a panel. Two regularities hold across backbones. First, γ has the same two regimes: WW-AG climbs toward large γ on DeepSeek-8B as on Qwen3.5-9B (38.62 at $\gamma = 0$ to 45.50 at $\gamma = 1.0$), while every other cell peaks at $\gamma \in \{0.1, 0.2\}$ and decays beyond, so the correction is a nudge where the base score is already strong and carries most of the signal where it is not. Second, the selected layer is the per-layer optimum of post-rescoring accuracy in seven of the eight cells (the exception is DeepSeek-8B TE-Mag, where block 9 edges the selected block 10 by 1.4 points); WW-HC concentrates in the last block on both backbones, and DeepSeek-8B consistently prefers the late attention band (24–32). These results suggest that the selected configurations sit on broad optima rather than on isolated peaks.

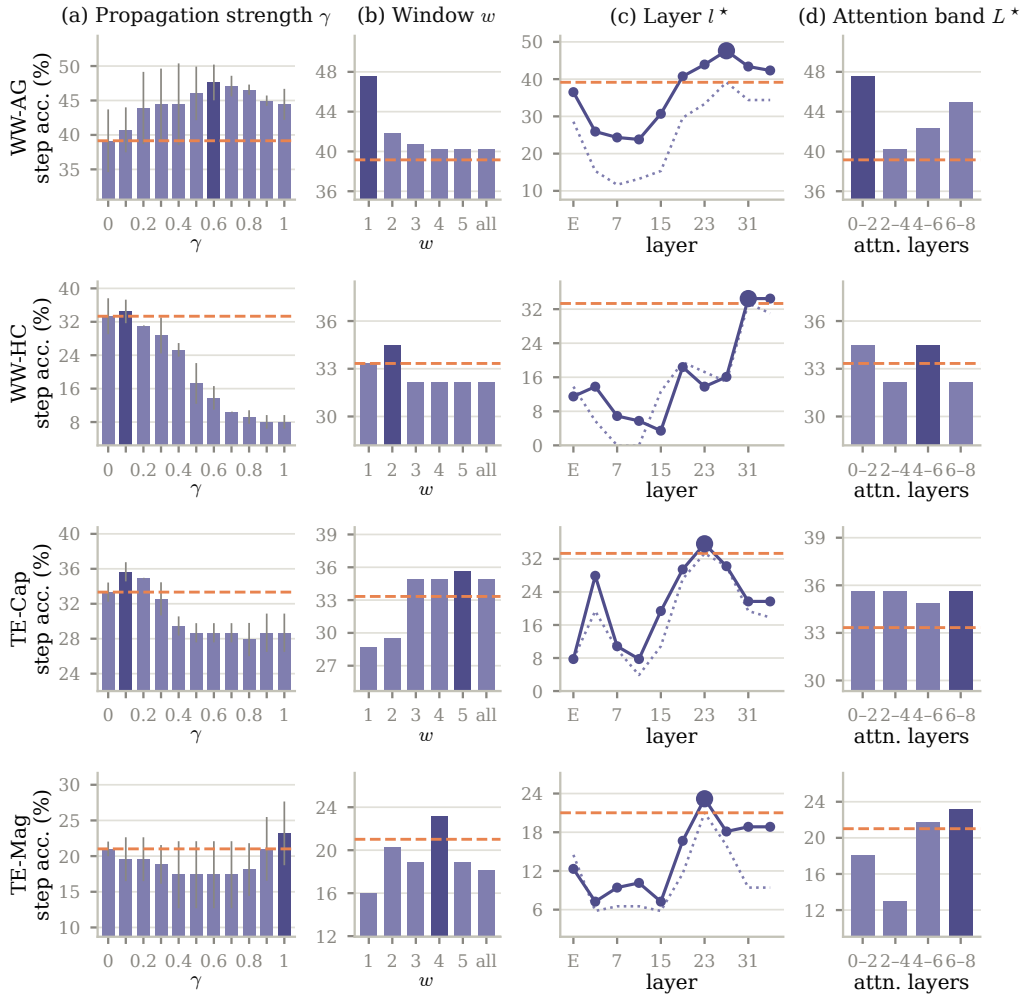


Figure 7: Sensitivity of SOAP on Qwen3.5-9B, all four subsets. As Figure 5, with the window w added: (a) propagation strength γ, (b) window w, (c) representation layer $l^\star$ (solid: after rescoring; dotted: base score at that layer; large marker: selected layer), (d) attention band $L^\star$. The dashed orange line denotes the base score of the selected configuration.

B.5 Pooling of the step representation

SOAP averages the hidden states over a step's tokens. We compare this with the last-token representation, the choice of OAT and StepFinder, by re-selecting the full configuration on last-token representations under the same rule, so each pooling stands on its best configuration. As shown

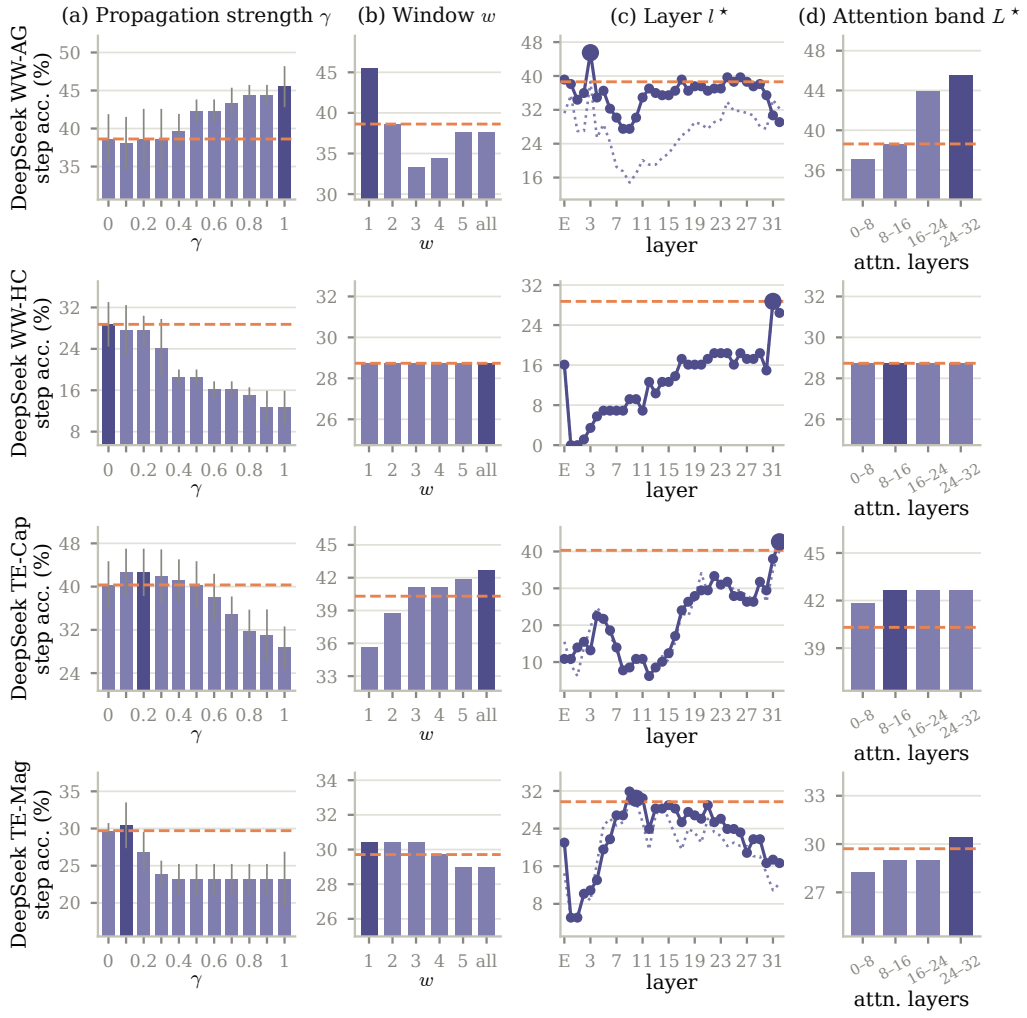


Figure 8: Sensitivity of SOAP on DeepSeek-R1-Distill-Llama-8B, all four subsets; panels as in Figure 7. DeepSeek-8B sweeps every block in panel (c). On WW-HC the selected γ is 0, so the w and attention-band panels are flat by construction.

in Table 13, mean pooling is clearly better: on WW-AG the base score drops from 39.15 to 29.10 on Qwen3.5-9B and from 38.62 to 29.10 on DeepSeek-8B, and on TE-Cap from 40.31 to 24.81 on DeepSeek-8B; only TE-Mag is close. Rescoring still helps on last-token representations (e.g., 29.10 to 30.69 on DeepSeek WW-AG), but from a much lower base. These results indicate that the spectral signal lives in the step's tokens as a whole rather than in the state at its final token, which mostly reflects what comes next.

Table 13: Mean vs. last-token pooling of the step representation. Step-level accuracy (%) with the full configuration re-selected per pooling; the best result in each column is highlighted in **bold**.

		Qwen3.5-9B				DeepSeek-8B			
Pooling	Method	WW-AG	WW-HC	TE-Cap	TE-Mag	WW-AG	WW-HC	TE-Cap	TE-Mag
Last token	Base	29.10	22.99	24.03	21.01	29.10	22.99	24.81	24.64
	SOAP	29.63	24.14	25.58	23.19	30.69	22.99	24.81	26.09
Mean (Ours)	Base	39.15	33.33	33.33	21.01	38.62	28.74	40.31	29.71
	SOAP	**47.62**	**34.48**	**35.66**	**23.19**	**45.50**	**28.74**	**42.64**	**30.43**

B.6 Quantity of unlabeled reference data

Table 14 completes Figure 5(d) with the remaining cells and DeepSeek-8B. With a third of the reference corpus (6–12 trajectories) every cell is within about four points of its full-data accuracy, and most within one or two. Because the configuration is re-selected at each fraction on the test split, a small fit can occasionally score above the full one (DeepSeek-8B WW-HC and TE-Mag at 10%); this is selection noise on tiny fits, not a real gain. These results confirm that a handful of unlabeled trajectories suffices to construct the reference.

Table 14: Quantity of unlabeled reference data, all cells. Step-level accuracy (%) before (S) and after (+SOAP) rescoring when the reference is fit on $\{10, 20, 30\}\%$ of the corpus, validation and test fixed, full configuration re-selected per fraction.

	WW-AG		WW-HC		TE-Cap		TE-Mag	
Reference data	S	+SOAP	S	+SOAP	S	+SOAP	S	+SOAP
Qwen3.5-9B								
10%	35.98	43.39	33.33	33.33	33.33	34.88	21.74	23.19
20%	38.10	44.44	33.33	33.33	33.33	34.88	21.74	23.19
30%	39.15	47.62	33.33	34.48	33.33	35.66	21.01	23.19
DeepSeek-8B								
10%	36.51	37.04	27.59	29.89	37.21	41.86	31.16	31.88
20%	38.62	46.03	28.74	28.74	39.53	42.64	30.43	30.43
30%	38.62	45.50	28.74	28.74	40.31	42.64	29.71	30.43

C Additional analysis

C.1 Agent-level accuracy and variance across seeds

Agent-level accuracy. Table 15 reports the responsible-agent accuracy of every row of Table 1, i.e., how often the agent of the predicted step is the annotated agent. The metric is far more forgiving than step-level accuracy because trajectories involve two to six agents (Table 6), so a method that names the right agent at a wrong step still scores. SOAP is best or second on every column of both open-backbone blocks (60.32 on WW-AG and 67.82 on WW-HC with Qwen3.5-9B; 61.90 and 65.52 with DeepSeek-8B), and rescoring never lowers it by more than a fraction of a point. The prompt-based judges close much of their step-level gap here: All-at-Once with Qwen3.5-9B reaches 81.40 on CE while locating the step in 33.47% of cases, which says that a judge often blames the right agent for the wrong reason. These results indicate that agent-level accuracy rewards the coarse verdict a judge produces naturally, whereas step-level accuracy measures localization.

Variance across seeds. Table 16 gives the standard deviation over the three seeds of every step-level number in Table 1. On the small subsets one trajectory is worth 1.6 points (WW-AG, 63 test trajectories) to 3.4 points (WW-HC, 29), so standard deviations of 2–5 points are the norm for every method, and differences below that on a single subset should not be read; CE, with over a thousand test trajectories, is stable to under one point. SOAP's margins over the strongest baseline on WW-AG (47.62 vs. 46.03 for RAFFLES with Qwen3.5-9B) are within one standard deviation, while its margins over the training-based methods and over every judge on DeepSeek-8B exceed several.

C.2 Accuracy at k

SOAP, OAT, and StepFinder each score every step of a trajectory, so the top-k steps form a ranked shortlist for triage; the prompt-based judges emit one verdict and are excluded here. We report step-level and agent-level accuracy at $k \in \{1, 3, 5\}$ and the mean reciprocal rank of the decisive step (MRR), all on the test splits of Table 1. For OAT and StepFinder we rank their stored per-step scores; a decisive step OAT never scored counts as a miss at every k.

As shown in Table 17, SOAP keeps its lead as the shortlist grows: at $k = 3$ it locates the decisive step in 74.60% of WW-AG trajectories with Qwen3.5-9B against 56.72 for OAT and 55.24 for StepFinder, and at $k = 5$ in 91.01% against 86.46 and 75.87; on WW-HC, whose trajectories average

Table 15: Agent-level accuracy (%) in the without-ground-truth setting, for every row of Table 1 and the GPT-5 judge. Mean over the seed triple. The best result in each column within a block is highlighted in **bold**.

Method	WW-AG	WW-HC	CE	TE-Cap	TE-Mag
GPT-4o					
All-at-Once	53.44	**59.77**	**78.86**	**67.44**	66.67
Step-by-Step	30.16	49.43	51.95	47.29	53.62
Binary Search	**58.73**	12.64	11.94	20.93	50.72
CORRECT	54.50	54.02	76.06	66.67	**76.09**
CHIEF	40.21	48.28	70.28	60.47	64.49
RAFFLES	**58.73**	51.72	78.82	62.79	60.14
GPT-5					
All-at-Once	**61.90**	45.98	80.62	63.57	68.12
Step-by-Step	46.56	47.13	70.45	50.39	65.22
Binary Search	59.79	60.92	78.54	56.59	69.57
CORRECT	58.73	50.57	**81.22**	62.79	69.57
CHIEF	52.38	**63.22**	–	51.94	63.04
RAFFLES	58.20	56.32	–	**65.12**	**74.64**
Qwen3.5-9B					
All-at-Once	56.61	48.28	**81.40**	62.79	62.32
Step-by-Step	19.05	47.13	35.85	51.16	65.94
Binary Search	38.10	62.07	70.70	22.48	59.42
CORRECT	59.79	47.13	78.60	62.02	**74.64**
CHIEF	50.26	54.02	74.81	**65.12**	**74.64**
RAFFLES	**60.85**	60.92	77.27	**65.89**	68.84
StepFinder	39.26	37.01	60.03	47.75	57.83
OAT	40.53	53.10	70.58	46.67	62.03
SOAP (w/o rescoring)	52.38	**67.82**	74.27	58.14	61.59
SOAP	60.32	**67.82**	74.06	60.47	65.22
DeepSeek-R1-Distill-Llama-8B					
All-at-Once	42.33	45.98	38.04	54.26	63.04
Step-by-Step	44.97	37.93	15.23	33.33	38.41
Binary Search	28.57	48.28	53.37	44.96	**69.57**
CORRECT	44.97	32.18	58.67	38.76	60.87
CHIEF	37.04	52.87	56.71	51.16	63.77
RAFFLES	38.10	51.72	54.38	54.26	65.94
StepFinder	48.99	41.15	67.29	45.12	42.61
OAT	42.33	56.32	69.04	50.08	63.91
SOAP (w/o rescoring)	58.73	**65.52**	75.72	63.57	62.32
SOAP	**61.90**	**65.52**	**75.84**	**65.12**	62.32

52 turns, the gap at $k = 5$ is 12 points over OAT (50.57 vs. 38.39). The baselines close in on the short CORRECT-Error trajectories, where five candidates cover most of a trajectory (Table 6). Rescoring helps most at $k = 1$ and $k = 3$, the ranks a triage would read first, and by $k = 5$ the base score and SOAP are within a point: the correction moves the decisive step to the top rather than reshuffling the tail. MRR summarizes the same picture, 63.92 vs. 41.46 for OAT on WW-AG. Agent-level accuracy at k (Table 18) saturates quickly for every method because a shortlist of three steps in a two- or three-agent trajectory names nearly every agent, so the step-level numbers are the informative ones. These results indicate that SOAP produces a better-ordered shortlist, not only a better top-1.

C.3 RESULTS WITH GPT-5 AS THE JUDGE

Table 19 repeats the prompt-based rows of Tables 1 and 2 with GPT-5 as the judge; the SOAP rows are those of the main tables. GPT-5 lifts every judge, and Binary Search most (41.80 on WW-AG without the answer, 75.71 on CE), so the judge's capacity rather than the prompting protocol sets these methods' accuracy. Even so, SOAP on a 9B or 8B backbone stays ahead of every GPT-5 judge on WW-AG, TE-Cap, and (on DeepSeek-8B) TE-Mag without the answer, and RAFFLES with GPT-5 overtakes it only on WW-HC (35.63 vs. 34.48) and, with the answer, on WW-AG (49.21 vs. 43.39); on CE GPT-5's Binary Search is clearly the best method. These results indicate that a

Table 16: Step-level accuracy (%) in the without-ground-truth setting as mean ± standard deviation over the three seeds of the frozen triple, for every row of Table 1 and the GPT-5 judge. OAT and StepFinder are first averaged over their five training seeds within each split.

Method	WW-AG	WW-HC	CE	TE-Cap	TE-Mag
GPT-4o					
All-at-Once	14.81 ±2.70	6.90 ±0.00	28.39 ±0.36	16.28 ±3.29	5.80 ±2.05
Step-by-Step	20.63 ±1.30	13.79 ±5.63	44.47 ±1.25	13.95 ±6.85	13.04 ±3.55
Binary Search	22.22 ±3.43	4.60 ±1.63	9.70 ±0.82	9.30 ±1.90	6.52 ±0.00
CORRECT	15.34 ±0.75	4.60 ±1.63	35.12 ±1.93	10.85 ±4.78	13.77 ±2.71
CHIEF	25.93 ±3.96	14.94 ±1.63	38.56 ±0.86	13.95 ±1.90	8.70 ±3.07
RAFFLES	42.86 ±3.43	25.29 ±6.50	64.22 ±0.64	24.81 ±1.10	17.39 ±3.07
GPT-5					
All-at-Once	18.52 ±0.75	4.60 ±1.63	64.15 ±1.29	10.85 ±3.95	10.87 ±1.77
Step-by-Step	24.34 ±0.75	18.39 ±1.63	64.38 ±0.57	17.83 ±1.10	6.52 ±0.00
Binary Search	41.80 ±0.75	21.84 ±1.63	75.71 ±0.72	24.03 ±2.90	20.29 ±1.02
CORRECT	24.87 ±4.55	13.79 ±2.82	35.20 ±0.52	20.93 ±0.00	23.91 ±3.07
CHIEF	27.51 ±1.98	25.29 ±5.86	–	22.48 ±3.95	16.67 ±4.47
RAFFLES	39.68 ±2.59	35.63 ±3.25	–	33.33 ±5.80	33.33 ±1.02
Qwen3.5-9B					
All-at-Once	17.46 ±2.59	11.49 ±4.30	33.47 ±0.53	4.65 ±1.90	1.45 ±2.05
Step-by-Step	10.58 ±1.50	16.09 ±1.63	31.15 ±2.46	14.73 ±2.90	10.87 ±4.70
Binary Search	2.65 ±0.75	13.79 ±5.63	59.92 ±0.88	0.78 ±1.10	5.07 ±3.69
CORRECT	29.10 ±2.99	3.45 ±0.00	36.35 ±0.46	7.75 ±1.10	2.90 ±2.71
CHIEF	28.57 ±2.24	3.45 ±0.00	29.39 ±0.65	3.88 ±1.10	2.17 ±1.77
RAFFLES	46.03 ±2.59	27.59 ±2.82	73.14 ±0.85	29.46 ±1.10	23.91 ±3.07
StepFinder	15.87 ±0.78	13.33 ±3.39	30.03 ±1.07	18.29 ±3.90	6.67 ±0.54
OAT	16.72 ±0.79	10.11 ±6.33	56.50 ±0.82	15.35 ±1.52	18.84 ±2.76
SOAP (w/o rescoring)	39.15 ±4.55	33.33 ±4.30	61.38 ±0.79	33.33 ±1.10	21.01 ±1.02
SOAP	47.62 ±2.59	34.48 ±2.82	61.78 ±0.73	35.66 ±1.10	23.19 ±4.47
DeepSeek-R1-Distill-Llama-8B					
All-at-Once	7.41 ±3.26	5.75 ±1.63	21.95 ±0.85	4.65 ±1.90	2.90 ±1.02
Step-by-Step	16.93 ±1.98	9.20 ±1.63	9.04 ±1.74	11.63 ±1.90	4.35 ±1.77
Binary Search	7.94 ±1.30	12.64 ±3.25	34.15 ±0.94	10.08 ±5.80	13.77 ±2.71
CORRECT	20.11 ±1.98	11.49 ±3.25	32.82 ±1.12	3.88 ±1.10	0.72 ±1.02
CHIEF	17.46 ±3.43	2.30 ±1.63	8.40 ±0.42	7.75 ±1.10	9.42 ±2.71
RAFFLES	28.04 ±1.98	8.05 ±3.25	41.60 ±0.71	20.16 ±2.90	10.87 ±3.55
StepFinder	18.94 ±2.84	10.80 ±3.10	36.91 ±0.22	14.73 ±0.79	4.20 ±1.08
OAT	12.70 ±2.99	15.86 ±8.29	52.50 ±1.13	17.98 ±0.22	13.04 ±1.98
SOAP (w/o rescoring)	38.62 ±3.26	28.74 ±4.30	64.19 ±0.25	40.31 ±4.39	29.71 ±1.02
SOAP	45.50 ±2.70	28.74 ±4.30	64.68 ±0.42	42.64 ±4.39	30.43 ±3.07

Table 17: Step-level accuracy at k (%) and mean reciprocal rank (MRR, %) in the without-ground-truth setting for the three methods that rank every step. The best result in each column within a block is highlighted in **bold**.

	WW-AG			WW-HC			CE			TE-Cap			TE-Mag			MRR
Method	@1	@3	@5	@1	@3	@5	@1	@3	@5	@1	@3	@5	@1	@3	@5	WW-AG
Qwen3.5-9B																
OAT	16.72	56.72	86.46	10.11	25.75	38.39	56.50	74.60	82.18	15.35	53.02	73.80	18.84	32.17	40.29	41.46
StepFinder	15.87	55.24	75.87	13.33	29.66	34.48	30.03	59.04	68.58	18.29	37.21	50.70	6.67	17.10	28.12	41.23
SOAP (w/o rescoring)	39.15	71.96	**92.59**	33.33	**43.68**	49.43	61.38	79.20	85.58	33.33	60.47	75.19	21.01	31.16	41.30	58.83
SOAP	**47.62**	**74.60**	91.01	**34.48**	42.53	**50.57**	**61.78**	**79.61**	**85.74**	**35.66**	**63.57**	**75.97**	**23.19**	**32.61**	**42.75**	**63.92**
DeepSeek-R1-Distill-Llama-8B																
OAT	12.70	45.19	83.92	15.86	37.01	42.30	52.50	74.08	83.17	17.98	47.13	75.04	13.04	31.01	41.45	37.62
StepFinder	18.94	60.32	78.94	10.80	21.61	29.20	36.91	63.05	74.12	14.73	41.24	58.60	4.20	11.01	21.45	43.75
SOAP (w/o rescoring)	38.62	72.49	91.53	**28.74**	**42.53**	**59.77**	64.19	79.96	87.01	40.31	58.14	75.97	29.71	34.78	**47.83**	58.54
SOAP	**45.50**	**76.19**	**93.12**	**28.74**	**42.53**	**59.77**	**64.68**	**80.30**	**87.41**	**42.64**	**62.02**	**78.29**	**30.43**	**39.13**	44.20	**63.71**

stronger judge narrows the gap on the short-trajectory corpus and on the answer-aided setting, while SOAP remains the most accurate method on the long agentic trajectories with no judge at all.

Table 18: Agent-level accuracy at k (%) in the without-ground-truth setting: a hit if any of the top-k steps belongs to the annotated agent. The best result in each column within a block is highlighted in **bold**.

	WW-AG			WW-HC			CE			TE-Cap			TE-Mag		
Method	@1	@3	@5	@1	@3	@5	@1	@3	@5	@1	@3	@5	@1	@3	@5
Qwen3.5-9B															
OAT	40.53	92.06	**99.47**	53.10	**81.61**	**91.72**	70.58	**85.99**	**90.24**	46.67	89.46	92.25	62.03	90.14	92.90
StepFinder	39.26	82.75	93.86	37.01	52.87	56.78	60.03	86.71	92.04	47.75	77.05	86.20	57.83	86.09	94.35
SOAP (w/o rescoring)	52.38	91.53	98.94	**67.82**	72.41	77.01	**74.27**	85.26	88.12	58.14	**90.70**	**93.02**	61.59	**92.03**	**96.38**
SOAP	**60.32**	**93.12**	97.35	**67.82**	70.11	78.16	74.06	85.16	88.14	**60.47**	88.37	**93.02**	**65.22**	90.58	**96.38**
DeepSeek-R1-Distill-Llama-8B															
OAT	42.33	86.98	**97.67**	56.32	**81.84**	**92.18**	69.04	86.22	**91.93**	50.08	88.84	95.97	**63.91**	90.14	**97.97**
StepFinder	48.99	83.39	93.76	41.15	54.48	60.92	67.29	**87.55**	91.98	45.12	81.71	88.99	42.61	90.72	**98.84**
SOAP (w/o rescoring)	58.73	**94.71**	**98.41**	**65.52**	73.56	81.61	75.72	86.04	90.28	63.57	**91.47**	**96.90**	62.32	**92.03**	94.93
SOAP	**61.90**	91.01	96.83	**65.52**	73.56	81.61	**75.84**	86.74	91.63	**65.12**	**91.47**	**96.90**	62.32	91.30	96.38

Table 19: Step-level accuracy (%) of the prompt-based methods with GPT-5 as the judge, without and with the ground-truth answer. CHIEF and RAFFLES were not run on CORRECT-Error under GPT-5. The SOAP rows repeat Tables 1 and 2. The best result in each column within a block is highlighted in **bold**.

Method	WW-AG	WW-HC	CE	TE-Cap	TE-Mag
Without GT					
All-at-Once	18.52	4.60	64.15	10.85	10.87
Step-by-Step	24.34	18.39	64.38	17.83	6.52
Binary Search	41.80	21.84	**75.71**	24.03	20.29
CORRECT	24.87	13.79	35.20	20.93	23.91
CHIEF	27.51	25.29	–	22.48	16.67
RAFFLES	39.68	**35.63**	–	33.33	**33.33**
SOAP, Qwen3.5-9B	**47.62**	34.48	61.78	35.66	23.19
SOAP, DeepSeek-8B	45.50	28.74	64.68	**42.64**	30.43
With GT					
All-at-Once	17.99	9.20	64.51	10.85	13.77
Step-by-Step	28.04	24.14	66.49	20.16	15.22
Binary Search	47.09	24.14	**77.42**	31.01	25.36
CORRECT	32.80	21.84	38.25	27.13	20.29
CHIEF	29.63	18.39	–	14.73	22.46
RAFFLES	**49.21**	**34.48**	–	36.43	28.99
SOAP, Qwen3.5-9B	43.39	**34.48**	60.59	36.43	21.01
SOAP, DeepSeek-8B	44.97	29.89	65.19	**42.64**	**36.96**

C.4 LARGER BACKBONES

Table 20 gives the numbers behind Figure 4(a–b), with the seed standard deviations and the baselines at every size, and adds Qwen3.5-4B for the baselines. Within the Qwen3.5 family SOAP holds its level from 9B to 27B on both subsets (47.62 vs. 44.97 on WW-AG, within one standard deviation; 34.48 on WW-HC at both sizes, with the same $\gamma = 0.1$ lift over an identical 33.33 base). The Qwen3-14B point is competitive on WW-AG (42.86) but 9 points lower on WW-HC, where its selection degenerates to a mid-stack block with a one-component band and a rescoring that changes nothing; because it also crosses model generations, this is a family effect as much as a size effect. Neither baseline scales: OAT stays within 14–20 on WW-AG and 10–17 on WW-HC from 4B to 27B, and StepFinder moves without a trend, with training-seed standard deviations of 2–7 points. These results confirm that the margin of Table 1 is intact at every size we could run.

C.5 CROSS-DISTRIBUTION TRANSFER

Figure 9 gives all four transfer grids: both backbones under the test-selected convention of Table 1 (the configuration for each source–target pair is chosen on the target's test split, exactly as the in-distribution numbers are) and under the validation-selected convention (chosen on the target's validation split, which for the cross setting pools the target's reference and validation files, since the target's reference split is not used for fitting). The diagonal of every grid is the in-distribution

Table 20: Step-level accuracy (%) by backbone size on Who&When, mean ± standard deviation (over the seed triple for SOAP; over the five training seeds for OAT and StepFinder). Qwen3-14B belongs to the previous model generation. SOAP was not run at 4B. The best result in each column is highlighted in **bold**.

Subset	Method	Qwen3.5-4B	Qwen3.5-9B	Qwen3-14B	Qwen3.5-27B
WW-AG	OAT	20.00 ± 1.4	16.72 ± 1.7	13.97 ± 1.7	20.42 ± 1.6
	StepFinder	15.34 ± 4.0	15.87 ± 3.4	22.65 ± 4.2	13.54 ± 2.3
	SOAP (w/o rescoring)	–	39.15 ± 5.6	40.21 ± 4.6	38.62 ± 7.5
	SOAP	–	47.62 ± 3.2	42.86 ± 5.7	44.97 ± 3.3
WW-HC	OAT	11.03 ± 1.0	10.11 ± 1.3	16.78 ± 2.2	12.41 ± 1.3
	StepFinder	10.80 ± 3.0	13.33 ± 5.4	14.02 ± 3.3	16.55 ± 7.0
	SOAP (w/o rescoring)	–	33.33 ± 5.3	25.29 ± 8.0	33.33 ± 2.0
	SOAP	–	34.48 ± 3.5	25.29 ± 8.0	34.48 ± 3.5

number of Table 1. Under the test convention most cross cells land within a few points of the diagonal, and a foreign reference can even match or beat the in-distribution one on DeepSeek-8B (WW-HC→TE-Cap ties 42.64; WW-HC→TE-Mag reaches 33.33 vs. 30.43). The validation convention restores the gap, the diagonal winning every column on both backbones, but the degradation is graded, not catastrophic (DeepSeek-8B WW-HC→WW-AG keeps 39.68 of 45.50).

Table 21 shows what happens without re-selection: freezing the source's whole configuration and applying it to the target collapses transfer (Qwen3.5-9B WW-AG→WW-HC: 3.45). These results indicate that what is distribution-specific is chiefly the hyperparameter configuration, not the spectral reference.

Table 21: Transfer with the source's configuration frozen. Step-level accuracy (%) of SOAP when the reference and the full configuration selected on the source subset (rows) are applied unchanged to the target subset (columns); the diagonal is the in-distribution number.

	Qwen3.5-9B, target				DeepSeek-8B, target			
Source	WW-AG	WW-HC	TE-Cap	TE-Mag	WW-AG	WW-HC	TE-Cap	TE-Mag
WW-AG	**47.62**	3.45	20.16	5.80	**45.50**	3.45	20.16	9.42
WW-HC	23.81	**34.48**	20.16	14.49	13.76	**28.74**	34.11	12.32
TE-Cap	12.70	13.79	**35.66**	20.29	15.87	21.84	**42.64**	10.14
TE-Mag	11.64	10.34	27.91	**23.19**	24.34	19.54	16.28	**30.43**

C.6 SYNTHETIC REFERENCES, FULL RESULTS

Table 22 completes Figure 4(d) with the base scores and DeepSeek-8B (generation details in Appendix A.8). Every synthetic cell lands within 1–7 points of its real-reference row, the best generator per cell within about one point on Qwen3.5-9B WW-AG (46.56 vs. 47.62) and above the real reference on DeepSeek-8B WW-HC (29.89 vs. 28.74), and every synthetic SOAP row stays far above the training-based baselines of Table 1. Rescoring keeps working on synthetic references, by up to 8.5 points over the base score (Qwen3.5-9B WW-AG, Qwen corpus), except where $\gamma = 0$ is selected. Neither generator dominates: the GPT-4o corpus wins Qwen3.5-9B on WW-AG and the Qwen3.5-9B corpus wins the other three cells. These results indicate that a cheap open-weight generator is a viable source of reference trajectories.

C.7 QUALITATIVE EXAMPLES

Figure 10 shows two more trajectories on which rescoring flips the prediction from wrong to right, one from each TraceElephant system; both flips hold on all three seeds of the triple. In Figure 10(a), a five-step TE-Cap trajectory scored with DeepSeek-8B, the extraction agent answers a question about the Book of Esther by looking up the wrong place (Susa instead of India) at step 2, and the orchestrator's final turn reports the wrong prime minister. The base score peaks on that final answer, the visible symptom; the dependency weights route its anomaly back to the extraction step, and SOAP's argmax moves from step 4 to the annotated step 2. In Figure 10(b), a thirteen-step TE-

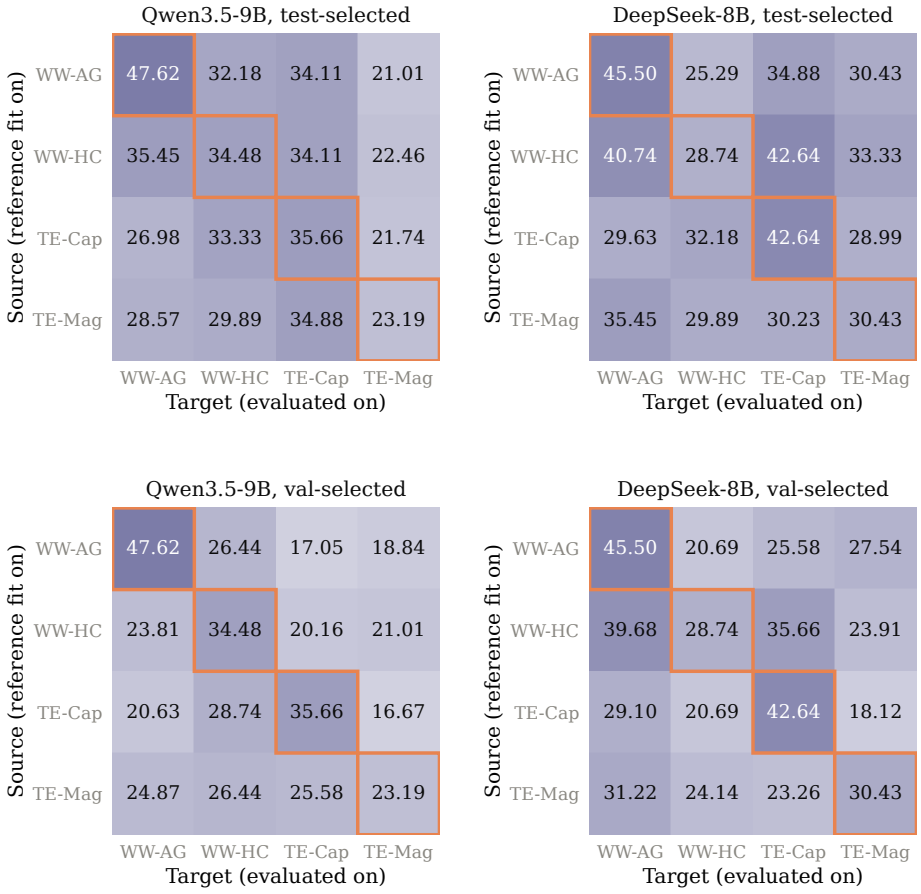
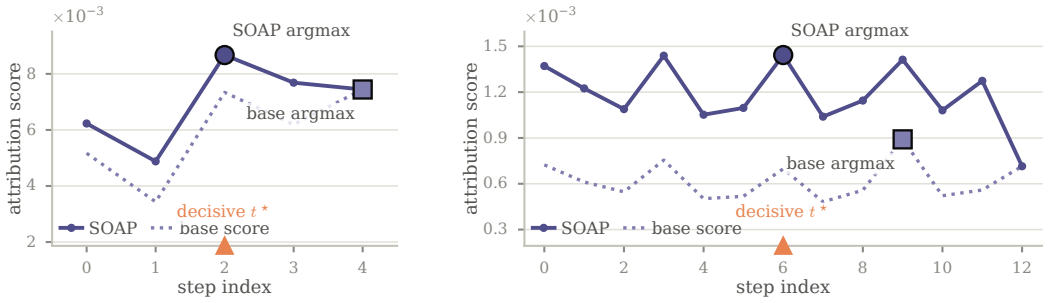


Figure 9: Cross-distribution transfer, all grids. Step-level accuracy (%) of SOAP with the reference fitted on the source subset (rows) and the configuration re-selected and evaluated on the target subset (columns), for both backbones, under the test-selected (top) and validation-selected (bottom) conventions. The outlined diagonal is the in-distribution number of Table 1 in every grid.

Table 22: Fitting the reference on real vs. synthetic trajectories. Step-level accuracy (%) before (S) and after (+SOAP) rescoring; only the reference corpus changes, the validation and test splits are those of Table 1.

	Qwen3.5-9B				DeepSeek-8B			
	WW-AG		WW-HC		WW-AG		WW-HC	
Reference	S	+SOAP	S	+SOAP	S	+SOAP	S	+SOAP
Real reference split	39.15	**47.62**	33.33	**34.48**	38.62	**45.50**	28.74	28.74
Synthetic, Qwen3.5-9B agents	33.33	41.80	28.74	31.03	35.98	39.68	26.44	**29.89**
Synthetic, GPT-4o agents	40.74	46.56	28.74	29.89	33.33	38.10	25.29	25.29

Mag trajectory scored with Qwen3.5-9B, the orchestrator fixes a conservative guessing strategy at step 6 that the annotation names as the decisive error; three turns later a code cell fails and the orchestrator's error report at step 9 becomes the base score's argmax. Rescoring lifts step 6, on which the failing turns depend, above the report of the failure they caused. Together, these examples show the mechanism the ablations measure: the correction moves blame from a late symptom to the step it was built on rather than shifting every score earlier.



(a) TE-Cap, DeepSeek-8B

(b) TE-Mag, Qwen3.5-9B

Figure 10: Per-step scores on two further TraceElephant trajectories, as in Figure 6: the base score (dotted) and SOAP (solid), the annotated decisive step marked in orange, and each score's argmax marked by the large symbol.